

Natural Language Processing (NLP)

Unit - 1

Introduction to NLP:

NLP in real world, language basics, approaches; Building NLP models – Eight steps; Web scraping.

Unit -2

Text Representation:

Vectorization, One-Hot, Bag of Words/N-Grams, TF-IDF, Word Embeddings, Doc2Vec; Text Classification workflow; Naïve Bayes.

Unit -3

Information Extraction:

Applications, Pipeline, Key-phrase extraction, Named Entity Recognition, Entity Disambiguation and Linking.

Unit - 4

NLP for Social Media:

Applications, Challenges, Processing social data, Twitter Sentiment, Meme & Fake News detection.

Unit -5

Chatbots:

Applications, Taxonomy, Dialog Systems; NLP for E-Commerce: Catalog search, Review & Sentiment analysis.

Text Books:

Daniel Jurafsky & James H. Martin, 'Speech and Language Processing', 3rd Ed., 2023.

Delip Rao & Brian McMahan, 'Natural Language Processing with PyTorch', O'Reilly, 2019.

Unit -2

Text Representation:

Vectorization, One-Hot, Bag of Words/N-Grams, TF-IDF, Word Embeddings, Doc2Vec; Text Classification workflow; Naïve Bayes.

Which of the following is **not** a typical real-world NLP application?

- a) Spam detection
- b) Chatbots
- c) Image compression
- d) Machine translation

Image compression

Autocomplete/predictive text on smartphones is an example of:

- a) Language modelling
- b) Web scraping
- c) Tokenization only
- d) Named entity linking

Language modelling

Which NLP approach relies on hand-crafted grammar rules and dictionaries?

- a) Statistical NLP
- b) Rule-based NLP
- c) Deep learning NLP
- d) Neural NLP

Rule-based NLP

Transformer-based models like BERT and GPT fall under which category of approach?

- a) Rule-based
- b) Statistical
- c) Deep learning
- d) Hybrid

Deep learning

The final step where the trained model is made available for real-world use (e.g., via API or app) is called:

- a) Training
- b) Deployment
- c) Cleaning
- d) Evaluation

Deployment

What is the purpose of the requests library in a typical web scraping workflow?

- a) Parse HTML tags
- b) Fetch web page content over HTTP
- c) Train ML models
- d) Tokenize text

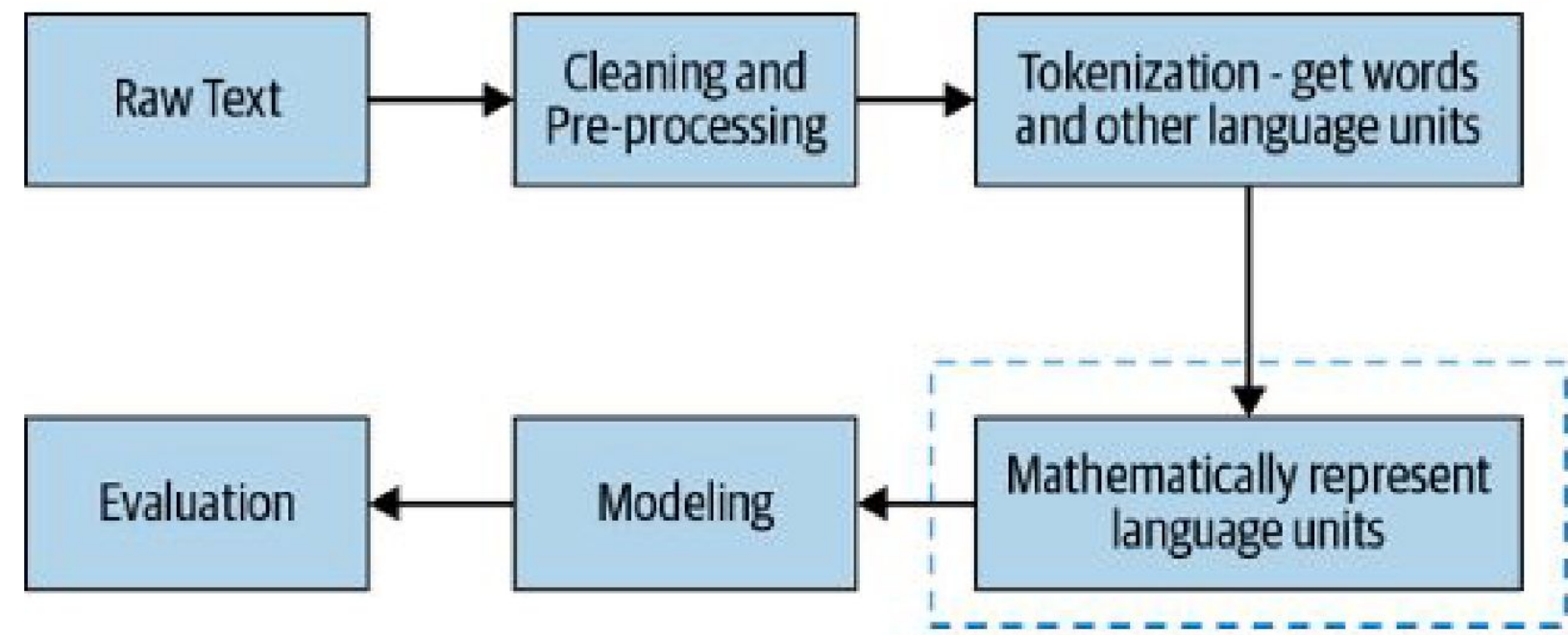
Fetch web page content over HTTP

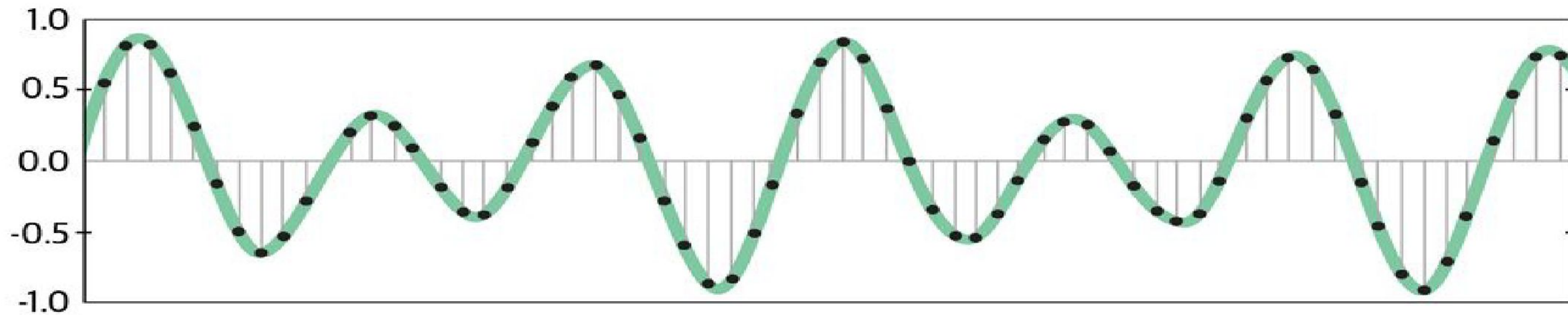
In the context of an NLP pipeline, web scraping is typically used at which stage?

- a) Model Evaluation
- b) Data Collection
- c) Deployment
- d) Feature Extraction

Data Collection

- To transform a given text into numerical form so that it can be fed into NLP and ML algorithms
- Conversion of raw text to a suitable numerical (numerical vector) form is called text representation





```
[-1274, -1252, -1160, -986, -792, -692, -614, -429, -286, -134, -57, -41,  
-169, -456, -450, -541, -761, -1067, -1231, -1047, -952, -645, -489, -448,  
-397, -212, 193, 114, -17, -110, 128, 261, 198, 390, 461, 772, 948, 1451,  
1974, 2624, 3793, 4968, 5939, 6057, 6581, 7302, 7640, 7223, 6119, 5461,  
4820, 4353, 3611, 2740, 2004, 1349, 1178, 1085, 901, 301, -262, -499,  
-488, -707, -1406, -1997, -2377, -2494, -2605, -2675, -2627, -2500, -2148,  
-1648, -970, -364, 13, 260, 494, 788, 1011, 938, 717, 507, 323, 324, 325,  
350, 103, -113, 64, 176, 93, -249, -461, -606, -909, -1159, -1307, -1544]
```

Features: ['enjoy', 'great', 'learning', 'love', 'nlp', 'is', 'i']

Vectorized text:

[[0 0 0 1 1 0 1] -

I love NLP

[0 1 0 0 1 1 0] -

NLP is great

[1 0 1 0 1 0 1]] -

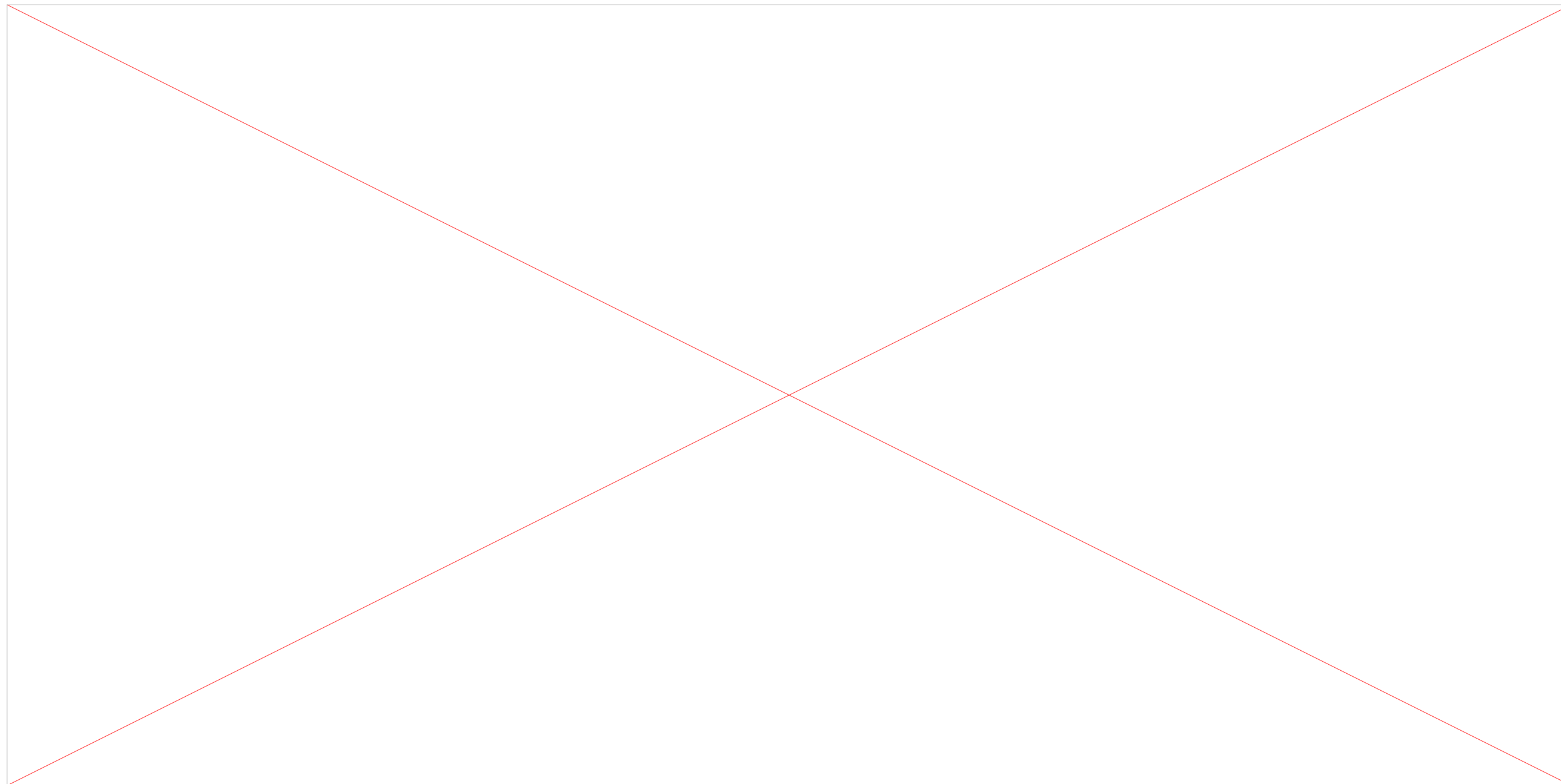
I enjoy learning NLP

The text data must be converted into some mathematical form

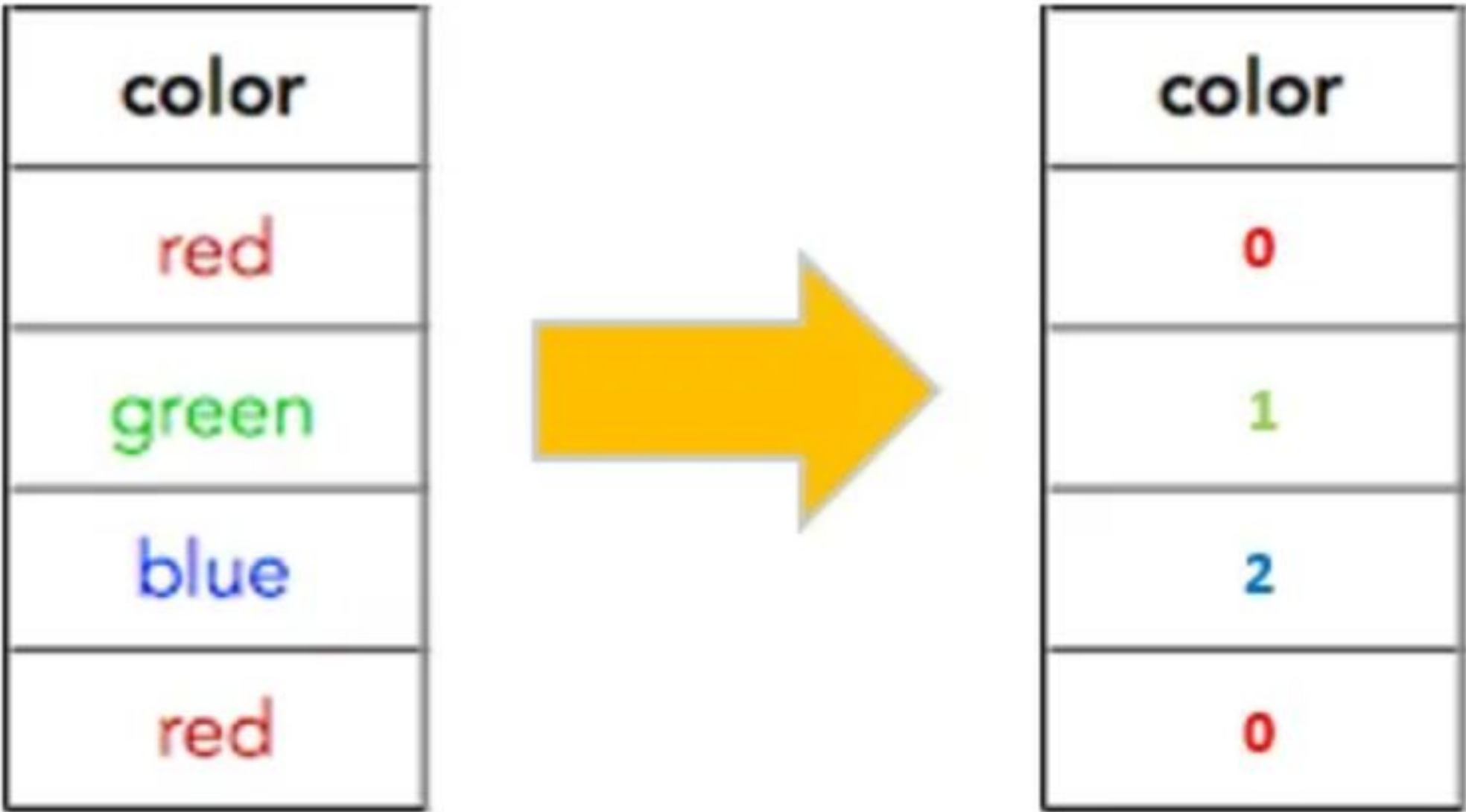
- It represent text units (characters, phonemes, words, phrases, sentences, paragraphs, and documents) with vectors of numbers
- This is known as the **vector space model (VSM)**
- VSM is a simple algebraic model used extensively for representing any text blob
- **Applications:** information-retrieval operations, document classification and document clustering
- It's a mathematical model that represents text units as vectors
- These are vectors of identifiers, such as index numbers in a corpus vocabulary

Activity – Guessing the Object

- Representing text data in a numerical format is a fundamental step in many natural language processing (NLP) tasks
- One of the basic approaches is to map each unique word in the vocabulary to a unique integer ID, and then represent each document as a vector of these IDs.
- Encoding is the process of converting data from one format into another format (Numbers)



Label encoding is a technique of converting categorical values inside columns into numerical ones.



Label encoding is a technique of converting categorical values inside columns into numerical ones.

```
from sklearn.preprocessing import LabelEncoder  
data = ['Red', 'Green', 'Blue', 'Red']  
le = LabelEncoder()  
encoded_data = le.fit_transform(data)  
print(f"Encoded Data: {encoded_data}")
```

Label encoding is a technique of converting categorical values inside columns into numerical ones.

sentence = "Label Encoding is important because many ML algorithms cannot process string values Categorical Data directly. Hence, converting them into numerical values is essential for model training. Categorical data is broadly divided into two types"

```
result = sentence.split ( sep = " " )  
from sklearn.preprocessing import LabelEncoder  
data = result  
le = LabelEncoder()  
encoded_data = le.fit_transform(data)  
print(f"Encoded Data: {encoded_data}")  
decoded_data = le.inverse_transform(encoded_data)  
print(f"Decoded Data: {decoded_data}")
```


One-Hot Encoding is a method used to convert categorical data into a numerical format.

```
from sklearn.preprocessing import OneHotEncoder
```

```
import numpy as np
```

```
words = np.array(["Label", "Encoding", "is", "important", "because", "many", "ML", "algorithms"]).reshape(-1, 1)
```

```
encoder = OneHotEncoder(sparse_output=False)
```

```
encoded = encoder.fit_transform(words)
```

```
print("Categories:", encoder.categories_[0])
```

```
print(encoded)
```

Categories: ['Encoding' 'Label' 'ML' 'algorithms' 'because' 'important' 'is' 'many']

```
[[0. 1. 0. 0. 0. 0. 0. 0.]  
 [1. 0. 0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0. 1. 0.]  
 [0. 0. 0. 0. 0. 1. 0. 0.]  
 [0. 0. 0. 0. 1. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0. 0. 1.]  
 [0. 0. 1. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 1. 0. 0. 0. 0.]]
```

The toy corpus with 4 documents:

- - D1: Dog bites man.
- - D2: Man bites dog.
- - D3: Dog eats meat.
- - D4: Man eats food.

Vocabulary from the toy corpus:

[dog, bites, man, eats, meat, food]

- **Text Processing**

Spam filters (Gmail, Outlook) historically used one-hot/BoW representations of words to classify emails as spam or not.

- **Computer Vision** - classifying images into categories
- **Healthcare / Medical Records** - blood type (A, B, AB, O), gender, or diagnosis codes (ICD-10 categories) are one-hot encoded

- Sparsity

One-hot encoded vectors are inherently sparse, with most of the elements being zero. This sparsity can lead to computational inefficiencies and make it difficult for machine learning.

- Fixed-Length Representation

One-hot encoding does not provide a fixed-length representation for text, as the length of the vector depends on the number of unique words in the corpus. This can be problematic for many machine learning algorithms that require inputs of a consistent size.

- Lack of Semantic Relationships

One-hot encoding treats words as completely independent entities, failing to capture any semantic relationships or similarities between them. This can limit the performance of models in tasks and understanding the meaning and context of words.

Bag of Words converts text into numbers that machine learning models can understand. It builds a vocabulary of all words in your dataset and then represents each document (sentence, paragraph, etc.) as a vector showing how many times each word appears.

Working of Bag of Words Model

The process of converting text into a bag of words involves:

- 1. Tokenization:** Divide the text into smaller units called tokens, usually words or phrases.
- 2. Counting word frequencies:** Create a vocabulary of all the unique words in the text corpus, and count the number of times each word appears in each document.
- 3. Encoding the data:** Encode the text data as numerical values by creating a vector for each document, with each element of the vector representing the frequency count of a particular word in the document.

Bag of Words Model explanation

Corpus
A collection of text documents



Tokenize
Divide the text into smaller units called tokens, usually words or phrases



Count word frequencies
Create a vocabulary of unique words and count the number of times each word appears in each document.



Encode the data
Encoding the text data as numerical values by creating a vector for each document, with each element of the vector representing the frequency count of a particular word in the document

© AIML.com Research

Example

Corpus:
The dog is happy. The child makes the dog happy. The dog makes the child happy



Tokenization:
D1: [The] [dog] [is] [happy]
D2: [The] [child] [makes] [the] [dog] [happy]
D3: [The] [dog] [makes] [the] [child] [happy]



Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1



Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

- Bag of words (BoW) is a classical text representation technique that has been used commonly in NLP, especially in text classification problems
- It involves converting a text into a set of words (or tokens) and then representing the text as a vector of word frequencies or binary indicators.
- The key idea behind it is as follows: represent the text under consideration as a bag (collection) of words while ignoring the order and context.
- The basic intuition behind it is that it assumes that the text belonging to a given class in the dataset is characterized by a unique set of words.
- If two text pieces have nearly the same words, then they belong to the same bag (class).
- Thus, by analyzing the words present in a piece of text, one can identify the class (bag) it belongs to.

- BoW maps words to unique integer IDs between 1 and $|V|$.
- Each document in the corpus is then converted into a vector of $|V|$ dimensions where in the i th component of the vector, is simply the number of times the word 'w' occurs in the document,
- i.e., we simply score each word in V by their occurrence count in the document.

S1 := “i am doing great as i was before”
S2 := “this is great”
S3 := “playing sport is a good habit”

	i	am	doing	great	as	was	before	this	good	used	to	playing	sport	is	a	habit
“i am doing great as i was before”	2	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
“this is great”	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0
“playing sport is a good habit”	0	0	0	0	0	0	0	0	1	0	0	1	1	1	1	1

Natural Language Processing (NLP)

Unit - 1

Introduction to NLP:

NLP in real world, language basics, approaches; Building NLP models – Eight steps; Web scraping.

Unit -2

Text Representation:

Vectorization, One-Hot, Bag of Words/N-Grams, TF-IDF, Word Embeddings, Doc2Vec; Text Classification workflow; Naïve Bayes.

Unit -3

Information Extraction:

Applications, Pipeline, Key-phrase extraction, Named Entity Recognition, Entity Disambiguation and Linking.

Unit - 4

NLP for Social Media:

Applications, Challenges, Processing social data, Twitter Sentiment, Meme & Fake News detection.

Unit -5

Chatbots:

Applications, Taxonomy, Dialog Systems; NLP for E-Commerce: Catalog search, Review & Sentiment analysis.

Text Books:

Daniel Jurafsky & James H. Martin, 'Speech and Language Processing', 3rd Ed., 2023.

Delip Rao & Brian McMahan, 'Natural Language Processing with PyTorch', O'Reilly, 2019.

Unit -2

Text Representation:

Vectorization, One-Hot, Bag of Words/N-Grams, TF-IDF, Word Embeddings, Doc2Vec; Text Classification workflow; Naïve Bayes.

1. Tossing a coin, Sample Space (S) = {H,T}
2. Rolling a die, Sample Space (S) = {1,2,3,4,5,6}
3. Find the probability of 'getting 3 on rolling a die'.

Total number of outcomes = $n(S) = 6$

Let A be the event of getting 3.

Number of favourable outcomes = $n(A) = 1$

i.e. $A = \{3\}$

Probability, $P(A) = n(A)/n(S) = 1/6$

Two dice are rolled, find the probability that the sum is:

equal to 1

equal to 4

less than 13

$S = \{ (1,1), (1,2), (1,3), (1,4), (1,5), (1,6)$
 $(2,1), (2,2), (2,3), (2,4), (2,5), (2,6)$
 $(3,1), (3,2), (3,3), (3,4), (3,5), (3,6)$
 $(4,1), (4,2), (4,3), (4,4), (4,5), (4,6)$
 $(5,1), (5,2), (5,3), (5,4), (5,5), (5,6)$
 $(6,1), (6,2), (6,3), (6,4), (6,5), (6,6) \}$

So, $n(S(\text{sample space})) = 36$

$$P(E) = n(E) / n(S) = 0 / 36 = 0$$

$$P(A) = n(A) / n(S) = 3 / 36 = 1 / 12$$

$$P(B) = n(B) / n(S) = 36 / 36 = 1$$

Conditional probability: Bayes' Theorem

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

There are two bags, In bag1 2 white ball and 3 red ball, In Bag2 5 white ball and 4 red ball. apply bayes theorem and find the probability of white ball in bag2

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

The probability that a randomly drawn white ball came from Bag 2 is **25 /43 (or approximately (0.581))**

Below is the step-by-step breakdown using **Bayes' Theorem**.

© Byjus.com

1. Identify the Given Information

- **Bag 1:** 2 White + 3 Red = **5 total balls**
- **Bag 2:** 5 White + 4 Red = **9 total balls**

2. Define the Events and Probabilities

Assuming both bags have an equal chance of being selected:

- Probability of choosing Bag 1: $(P(B1) = 1 / 2)$
- Probability of choosing Bag 2: $(P(B2) = 1 / 2)$

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Conditional probabilities of drawing a white ball from each bag:

- Probability of a white ball from Bag 1: $P(W/B_1) = 2/5$
- Probability of a white ball from Bag 2: $P(W/B_2) = 5/9$

© Byjus.com

3. Calculate Total Probability of Drawing a White Ball ($P(W)$)

Using the Law of Total Probability:

$$P(W) = [P(B_1) \times P(W|B_1)] + [P(B_2) \times P(W|B_2)]$$

$$P(W) = \frac{1}{5} + \frac{5}{18} = \frac{18 + 25}{90} = \frac{43}{90}$$

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Apply Bayes' Theorem

To find the probability that the ball came from Bag 2 *given* it is white

© Byjus.com

$$P(B_2|W) = \frac{P(B_2) \times P(W|B_2)}{P(W)}$$

$$P(B_2|W) = \frac{\frac{5}{18}}{\frac{43}{90}} = \frac{5}{18} \times \frac{90}{43} = \frac{25}{43}$$

Conditional probability: Bayes' Theorem

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

© Byjus.com

Applications

- **AI & Machine Learning** → Used in Naïve Bayes classifiers to predict outcomes.
- **Medical Testing** → Finding the real probability of having a disease after a positive test.
- **Spam Filters** → Checking if an email is spam based on keywords.
- **Weather Prediction** → Updating the chance of rain based on new data.

AI is transforming Industries - Tech

Quantum computing is the future - Tech

New smartphone released today -Tech

Football match was exciting – Non- Tech

New movie breaking records – Non-Tech

Cooking shows are popular- Non -Tech

Test : AI is the future of the company

Bayes Theorem - Example

Word	Tech	Non Tech
AI is transforming Industries	Tech	
Quantum computing is the future	Tech	
New smartphone released today	Tech	
Football match was exciting		Non Tech
New movie breaking records		Non Tech
Cooking shows are popular		Non Tech

Step 2: Priors

- $P(\text{Tech}) = 3/6 = 0.5$
- $P(\text{Non-Tech}) = 3/6 = 0.5$

Step 3: Word Counts per Class

Class	Total words	Vocabulary size ($ V $)
Tech	13	23
Non-Tech	12	23

Step 4: Likelihoods

$P(\text{word} | \text{class}) = (\text{count}(\text{word}, \text{class}) + 1 / \text{TotalWordsclass} + |V|$

Word	Count in Tech	$P(\text{word} \text{Tech})$	Count in Non-Tech	$P(\text{word} \text{Non-Tech})$
ai	1	$(1+1)/36 = 0.0556$	0	$(0+1)/35 = 0.0286$
is	2	$(2+1)/36 = 0.0833$	0	$1/35 = 0.0286$
the	1	$(1+1)/36 = 0.0556$	0	$1/35 = 0.0286$
future	1	$(1+1)/36 = 0.0556$	0	$1/35 = 0.0286$
of	0	$1/36 = 0.0278$	0	$1/35 = 0.0286$
company	0	$1/36 = 0.0278$	0	$1/35 = 0.0286$

Step 5: Apply Bayes' Theorem

$$P(Tech | text) \propto P(Tech) \times \prod P(word_i | Tech)$$

Tech:

$$0.5 \times 0.0556 \times 0.0833 \times 0.0556 \times 0.0556 \times 0.0278 \times 0.0278 \approx 5.51 \times 10^{-9}$$

Non-Tech:

$$0.5 \times 0.0286^6 \approx 2.72 \times 10^{-10}$$

Step 6: Decision

Class	Score
Tech	5.51×10^{-9} ✓
Non-Tech	2.72×10^{-10}

Bayes Theorem - Example 2

Doc	Text	Class
D1	Win money now	Spam
D2	Claim your free prize	Spam
D3	Limited offer win cash	Spam
D4	Meeting scheduled tomorrow	Ham
D5	Please review the report	Ham
D6	Lunch with team tomorrow	Ham

Test: "Win free money"

$$P(\text{Spam}) = 3/6 = 0.5, P(\text{Ham}) = 3/6 = 0.5$$

- Spam vocabulary words: win, money, claim, your, free, prize, limited, offer, cash (**9 words total**)
- Ham vocabulary words: meeting, scheduled, tomorrow, please, review, the, report, lunch, with, team (**10 words total**)
- $|V| = 17$ unique words

Word	P(word Spam) w/ smoothing	P(word Ham) w/ smoothing
win	$(1+1)/(9+17) = 0.0769$	$(0+1)/(10+17) = 0.0370$
free	$(1+1)/26 = 0.0769$	$(0+1)/27 = 0.0370$
money	$(1+1)/26 = 0.0769$	$(0+1)/27 = 0.0370$

$\text{Score}(\text{Spam}) = 0.5 \times 0.0769 \times 0.0769 \times 0.0769 \approx \mathbf{2.27 \times 10^{-4}} = 0.000227$
 $\text{Score}(\text{Ham}) = 0.5 \times 0.0370 \times 0.0370 \times 0.0370 \approx \mathbf{2.53 \times 10^{-5}} = 0.0000253$

→ Predicted: Spam 

Doc	Text	Class
D1	This movie is great	Positive
D2	Amazing acting and story	Positive
D3	I loved this film	Positive
D4	This movie is boring	Negative
D5	Terrible acting and story	Negative
D6	I hated this film	Negative

Test: *"I loved the acting"*

- $P(\text{Positive}) = 0.5$, $P(\text{Negative}) = 0.5$
- Positive words total = 12, Negative words total = 12, $|V| = 15$

Word	$P(\text{word} \text{Positive})$	$P(\text{word} \text{Negative})$
i	$(1+1)/27 = 0.0741$	$(1+1)/27 = 0.0741$
loved	$(1+1)/27 = 0.0741$	$(0+1)/27 = 0.0370$
acting	$(1+1)/27 = 0.0741$	$(1+1)/27 = 0.0741$

$$\text{Score}(\text{Positive}) = 0.5 \times 0.0741 \times 0.0741 \times 0.0741 \approx \mathbf{2.03 \times 10^{-4}}$$

$$\text{Score}(\text{Negative}) = 0.5 \times 0.0741 \times 0.0370 \times 0.0741 \approx \mathbf{1.01 \times 10^{-4}}$$

TF-IDF stands for term frequency-inverse document frequency, and it is a commonly used weighting scheme in natural language processing and information retrieval

The term frequency (TF) measures how often a word appears in a document

TF-IDF is used to measure the importance of a word in a document relative to a corpus of documents.

The TF-IDF weight for a word in a document is calculated by multiplying its term frequency by its inverse document frequency.

$$TF = \frac{\text{Number of times the terms appears in the document}}{\text{Total number of terms in the document}}$$

Sentence 1 = “I like to be an Engineer”

Sentence 2 = “I like to play cricket

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

$TF = \frac{\text{Number of times the terms appears in the document}}{\text{Total number of terms in the document}}$

Sentence 1 = “I like to be an Engineer”
Sentence 2 = “I like to play cricket”

Term	Count in S1	TF (S1)	Count in S2	TF (S2)
I	1	1/6 = 0.167	1	1/5 = 0.20
like	1	1/6 = 0.167	1	1/5 = 0.20
to	1	1/6 = 0.167	1	1/5 = 0.20
be	1	1/6 = 0.167	0	0.00
an	1	1/6 = 0.167	0	0.00
Engineer	1	1/6 = 0.167	0	0.00
play	0	0.00	1	1/5 = 0.20
Cricket	0	0.00	1	1/5 = 0.20

IDF (inverse document frequency) measures the importance of the term across a corpus.

- In computing TF, all terms are given equal importance (weightage).
- However, it's a well-known fact that stop words like is, are, am, etc., are not important, even though they occur frequently.
- To account for such cases, IDF weighs down the terms that are very common across a corpus and weighs up the rare terms.

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

For each word, count the number of documents in the corpus that contain the word. Calculate the logarithm of the total number of documents in the corpus divided by the number of documents containing the word to get the inverse document frequency

$$IDF = \log \frac{\text{number of document in the corpus}}{\text{number of documents the corpus which contain that term}}$$

TF-IDF score

- The TF-IDF score is a product of these two terms.

$$\text{TF-IDF score} = \text{TF} * \text{IDF}.$$

Inverse Document Frequency (IDF)

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

Term	df (docs containing term)	IDF = $\log_{10}(N/df) = \log_{10}(2/df)$
I	2	$\log_{10}(2/2) = \log_{10}(1) = 0$
like	2	$\log_{10}(2/2) = 0$
to	2	$\log_{10}(2/2) = 0$
be	1	$\log_{10}(2/1) = 0.301$
an	1	$\log_{10}(2/1) = 0.301$
Engineer	1	$\log_{10}(2/1) = 0.301$
play	1	$\log_{10}(2/1) = 0.301$
Cricket	1	$\log_{10}(2/1) = 0.301$

Sentence 1 = "I like to be an Engineer"
Sentence 2 = "I like to play cricket"

Calculating TF -IDF

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

Term	TF (S1)	TF (S2)	IDF	TF-IDF (S1)	TF-IDF (S2)
I	0.167	0.20	0	0.000	0.000
like	0.167	0.20	0	0.000	0.000
to	0.167	0.20	0	0.000	0.000
be	0.167	0.00	0.301	0.0502	0.000
an	0.167	0.00	0.301	0.0502	0.000
Engineer	0.167	0.00	0.301	0.0502	0.000
play	0.00	0.20	0.301	0.000	0.0602
Cricket	0.00	0.20	0.301	0.000	0.0602

Final TF-IDF Vectors

- **S1** = [I: 0, like: 0, to: 0, be: 0.0502, an: 0.0502, Engineer: 0.0502, play: 0, Cricket: 0]
- **S2** = [I: 0, like: 0, to: 0, be: 0, an: 0, Engineer: 0, play: 0.0602, Cricket: 0.0602]

Calculating TF -IDF

- D1: Dog bites man.
- D2: Man bites dog.
- D3: Dog eats meat.
- D4: Man eats food.

find TF and TF-IDF for the statements

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

Calculating TF -IDF

Step 1: Term Frequency (TF) Table

Term	D1	D2	D3	D4
bites	0.333	0.333	0.000	0.000
dog	0.333	0.333	0.333	0.000
eats	0.000	0.000	0.333	0.333
food	0.000	0.000	0.000	0.333
man	0.333	0.333	0.000	0.333
meat	0.000	0.000	0.333	0.000

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

- D1: Dog bites man.
- D2: Man bites dog.
- D3: Dog eats meat.
- D4: Man eats food.

find TF and TF-IDF for the statements

Calculating TF -IDF

Step 2: Document Frequency (df) and IDF

N = 4 documents. $IDF = \log_{10}(N / df)$

Term	df	$IDF = \log_{10}(4/df)$
bites	2	$\log_{10}(2) = 0.3010$
dog	3	$\log_{10}(1.333) = 0.1249$
eats	2	$\log_{10}(2) = 0.3010$
food	1	$\log_{10}(4) = 0.6021$
man	3	$\log_{10}(1.333) = 0.1249$
meat	1	$\log_{10}(4) = 0.6021$

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

- D1: Dog bites man.
- D2: Man bites dog.
- D3: Dog eats meat.
- D4: Man eats food.

find TF and TF-IDF for the statements

Calculating TF -IDF

Step 3: TF-IDF = TF × IDF

Term	D1	D2	D3	D4
bites	0.1003	0.1003	0.0000	0.0000
dog	0.0416	0.0416	0.0416	0.0000
eats	0.0000	0.0000	0.1003	0.1003
food	0.0000	0.0000	0.0000	0.2007
man	0.0416	0.0416	0.0000	0.0416
meat	0.0000	0.0000	0.2007	0.0000

Final TF-IDF Vectors

- **D1** = [bites: 0.1003, dog: 0.0416, eats: 0, food: 0, man: 0.0416, meat: 0]
- **D2** = [bites: 0.1003, dog: 0.0416, eats: 0, food: 0, man: 0.0416, meat: 0]
- **D3** = [bites: 0, dog: 0.0416, eats: 0.1003, food: 0, man: 0, meat: 0.2007]
- **D4** = [bites: 0, dog: 0, eats: 0.1003, food: 0.2007, man: 0.0416, meat: 0]

- **t** – term (word).
- **d** – document (set of words).
- **N** – count of corpus.
- **corpus** – the total document set.

- D1: Dog bites man.
- D2: Man bites dog.
- D3: Dog eats meat.
- D4: Man eats food.

find TF and TF-IDF for the statements

Natural Language Processing (NLP)

Unit - 1

Introduction to NLP:

NLP in real world, language basics, approaches; Building NLP models – Eight steps; Web scraping.

Unit -2

Text Representation:

Vectorization, One-Hot, Bag of Words/N-Grams, TF-IDF, Word Embeddings, Doc2Vec; Text Classification workflow; Naïve Bayes.

Unit -3

Information Extraction:

Applications, Pipeline, Key-phrase extraction, Named Entity Recognition, Entity Disambiguation and Linking.

Unit - 4

NLP for Social Media:

Applications, Challenges, Processing social data, Twitter Sentiment, Meme & Fake News detection.

Unit -5

Chatbots:

Applications, Taxonomy, Dialog Systems; NLP for E-Commerce: Catalog search, Review & Sentiment analysis.

Text Books:

Daniel Jurafsky & James H. Martin, 'Speech and Language Processing', 3rd Ed., 2023.

Delip Rao & Brian McMahan, 'Natural Language Processing with PyTorch', O'Reilly, 2019.

Unit -2

Text Representation:

Vectorization, One-Hot, Bag of Words/N-Grams, TF-IDF, Word Embeddings, Doc2Vec; Text Classification workflow; Naïve Bayes.

N-gram model is a type of Language Model(LM) that uses a simple, but powerful, mathematical concept to predict the probability of the next word in a sequence.

The N-gram model relies on the Markov Assumption, which states that the probability of the next word depends only on the immediate N-1 words.

Combining consecutive N words we could extract much more information from the documents.

Types of N – Grams

Unigram

Bigram

Trigram

Types of N – Grams

Unigram – one token

Bigram – Two token

Trigram – Three token

Example – Repeated task makes perfect

Finding Probability of the words for Bigram

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}.$$

Finding Probability of the words for Bigram

$$P(w_t | w_{t-2}, w_{t-1}) = \frac{\text{count}(w_{t-2}, w_{t-1}, w_t)}{\text{count}(w_{t-2}, w_{t-1})}.$$

Finding Probability of the words for Bigram

- "I like NLP"
- "I like AI"
- "I love NLP"

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

Finding Probability of the words for Bigram

- "I like NLP"
- "I like AI"
- "I love NLP"

Step 1: Add sentence boundary tokens (standard practice in bigram language models)

<s> I like NLP </s>

<s> I like AI </s>

<s> I love NLP </s>

Step 2: Extract all bigrams

Step 3: Bigram Frequency Count

Finding Probability of the words for Bigram

Step 2: Extract all bigrams

Sentence	Bigrams
1	(<s>,I), (I,like), (like,NLP), (NLP,</s>)
2	(<s>,I), (I,like), (like,AI), (AI,</s>)
3	(<s>,I), (I,love), (love,NLP), (NLP,</s>)

Step 3: Bigram Frequency Count

Bigram	Count
(<s>, I)	3
(I, like)	2
(I, love)	1
(like, NLP)	1
(like, AI)	1
(love, NLP)	1
(NLP, </s>)	2
(AI, </s>)	1

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

Finding Probability of the words for Bigram

Bigram Probability — this is where it connects to language modeling:

$$P(w_i | w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$$

Example: What's the probability the next word is "like" given the previous word is "I"?

- $\text{Count}(I, \text{like}) = 2$
- $\text{Count}(I) = 3$ (appears in all 3 sentences)

$$P(\text{like} | I) = \frac{2}{3} = 0.667$$

$$P(\text{like} | I) = 0.667$$

Similarly:

$$P(\text{love} | I) = \frac{1}{3} = 0.333$$

$$P(\text{love} | I) = 0.333$$

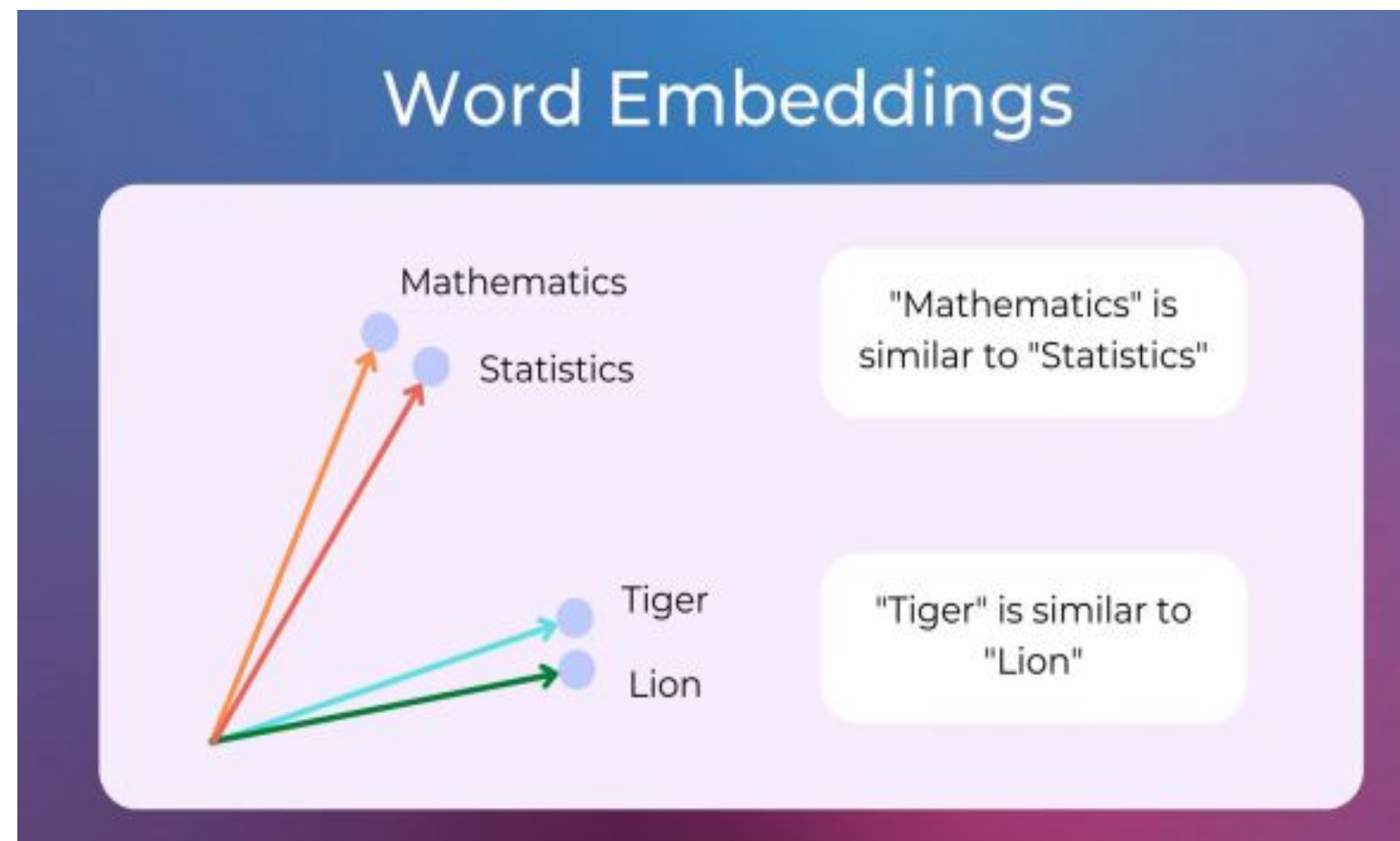
Word embedding is a technique used in natural language processing (NLP) that represents words as numbers so that a computer can work with them

Word embeddings have become integral to tasks such as text classification, sentiment analysis, machine translation.

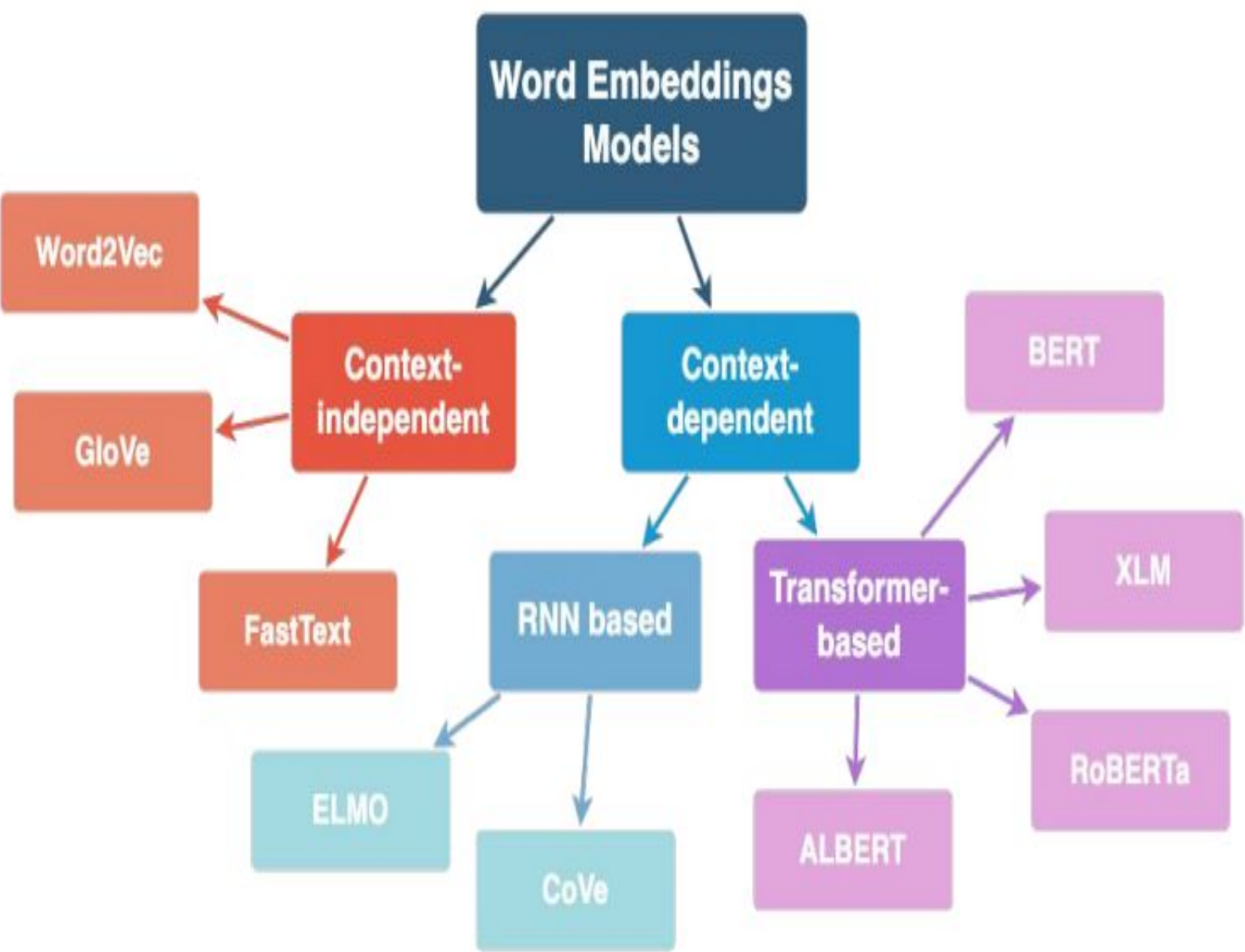
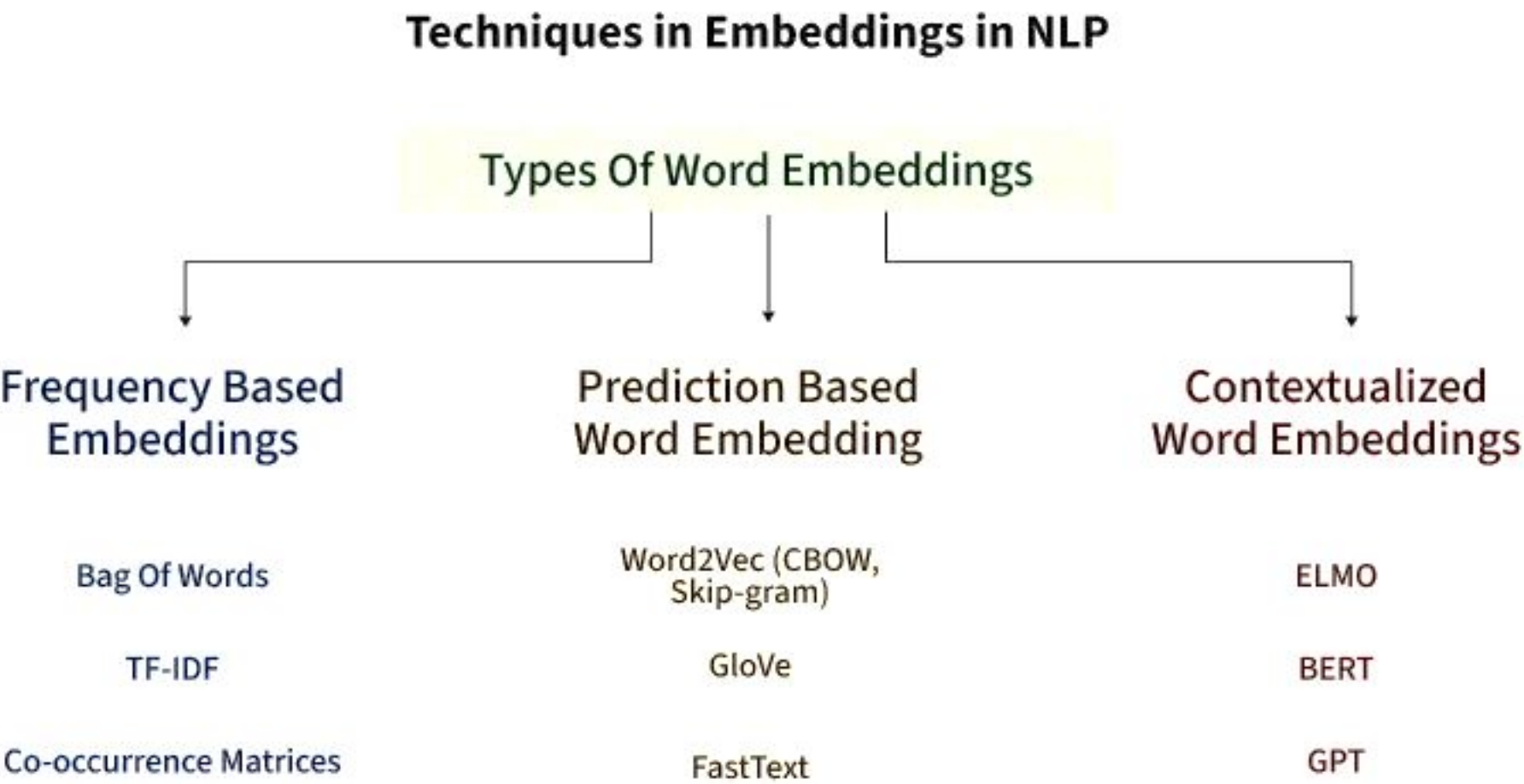
- Its Similar to BoW, TF-IDF vectors to calculate similarity between two texts using a similarity.
- TF-IDF is a commonly used representation in application scenarios such as information retrieval and text classification.
- However, despite the fact that TF-IDF is better than the vectorization methods in terms of capturing similarities between words, it still suffers from the curse of high dimensionality.

A downside of TF-IDF (and of bag of words approaches in general) is that the semantics of the tokens is not taken into account.

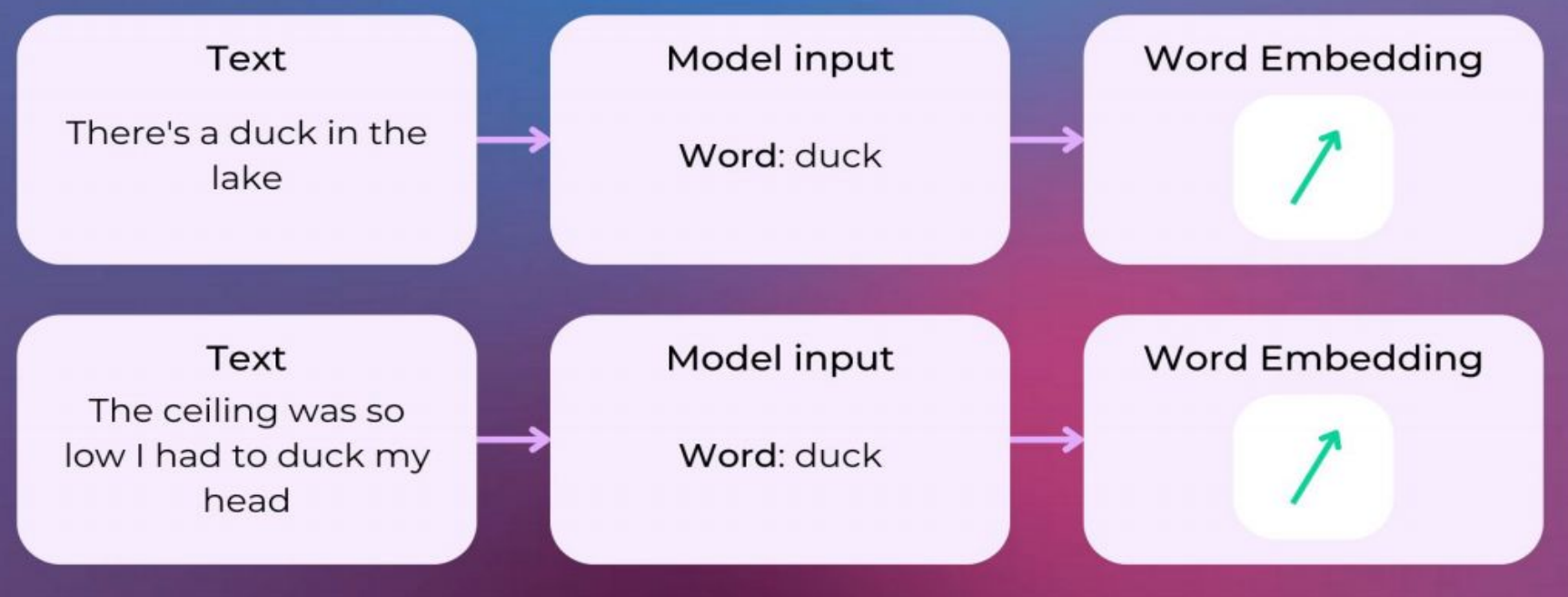
It would be great to represent tokens in a way so that semantically similar tokens have similar representations: we can do that with word embeddings



- Semantic Similarity: Embeddings allow for capturing semantic relationships between words. For example, the words "dog," "puppy," and "cat" might be close to each other in the vector space.
- Dimensionality Reduction: Unlike one-hot encoding, which results in sparse and high-dimensional vectors, embeddings provide a dense, lower-dimensional representation.
- Transfer Learning: Pre-trained embeddings can be used in various NLP tasks without training from scratch, saving computational resources and improving performance.



Context-Independent Word Embedding



- With context-independent embeddings models, the learned representations are characterized by being unique and distinct for each word, without considering the word's context.
- This means that homonyms like “duck” (which can be the animal or a verb) won't be disambiguated by their context (i.e. the text in which the word appears) and will always have a unique word embedding that somehow captures both meanings.
- As a result, these models produce a map between all the words in the vocabulary and their vectors.

- Word2Vec:
- Developed by Google.
- Two main models:

CBOW (Continuous Bag of Words) and Skip-gram.

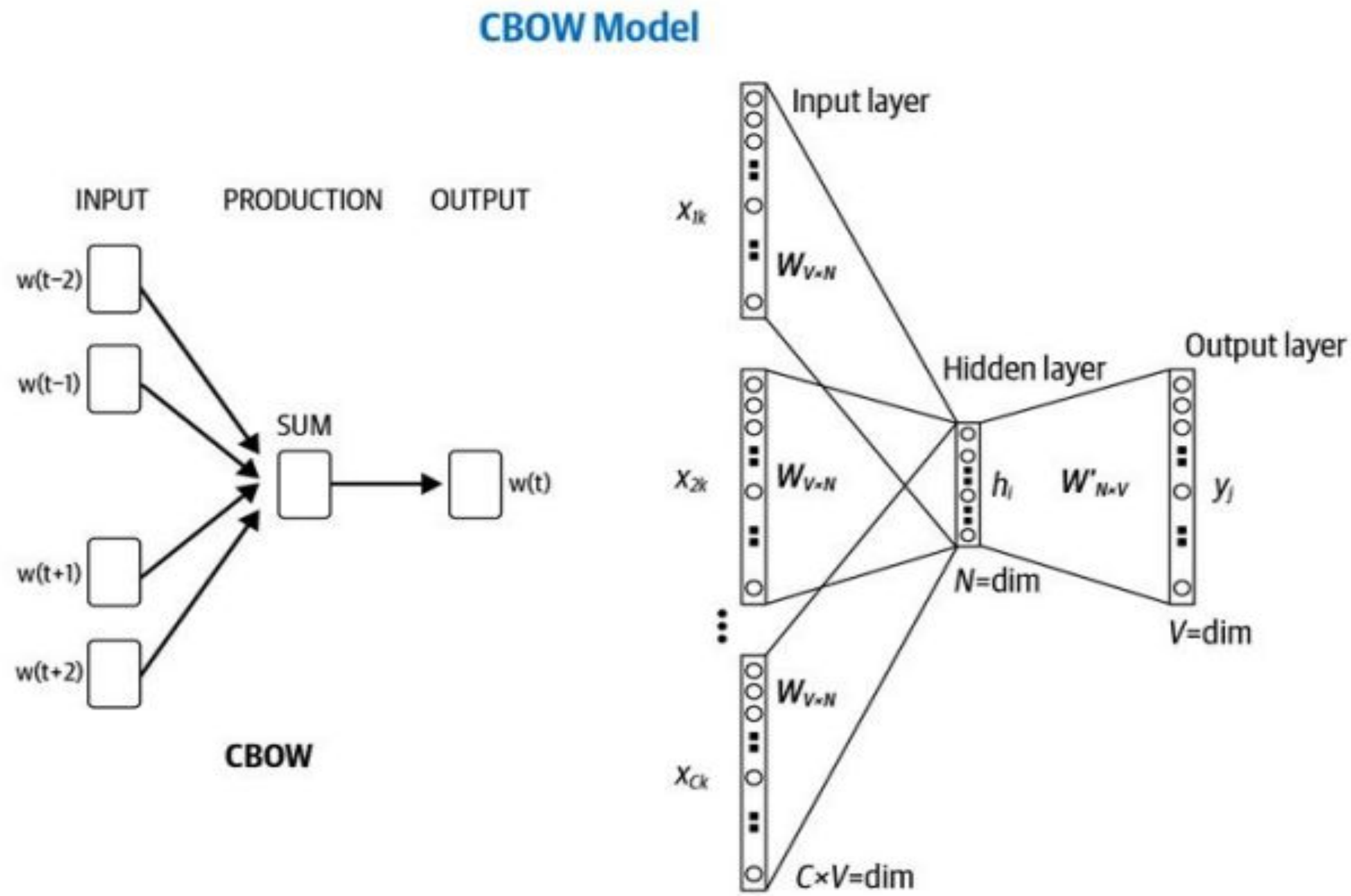
- CBOW (Continuous Bag of Words): Predicts the current word based on the context (surrounding words).
- Skip-gram: Predicts the surrounding words given the current word.
- Learns word vectors by predicting a word from its context (CBOW) or the context from a word (Skip-gram)

Continuous Bag of Words (CBOW) is a popular natural language processing technique used to generate word embeddings. Word embeddings are important for many NLP tasks because they capture semantic and syntactic relationships between words in a language.

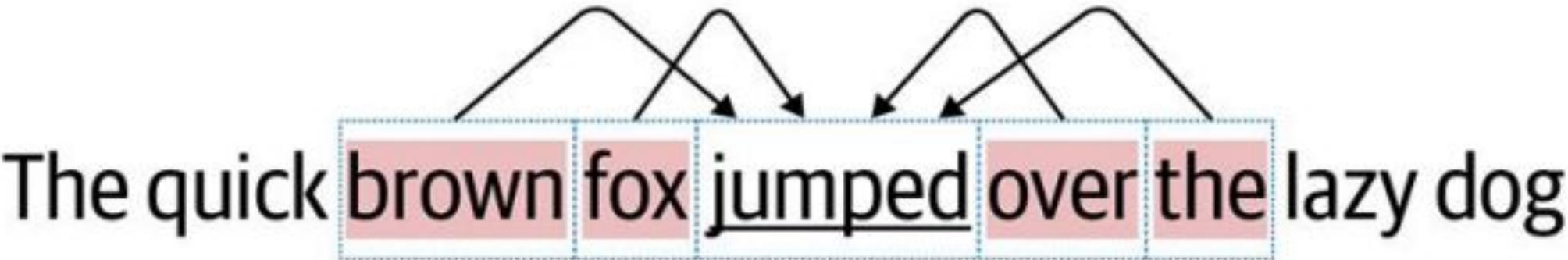
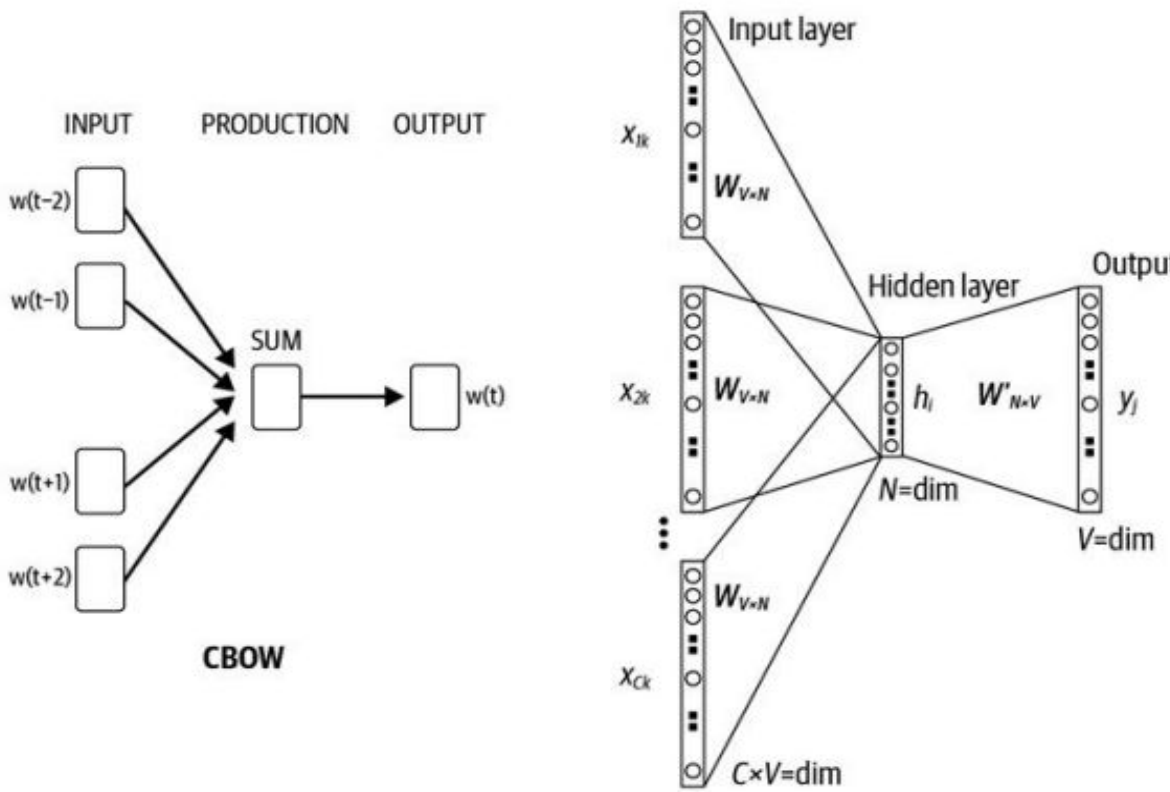
CBOW is a neural network-based algorithm that predicts a target word given its surrounding context words.

In CBOW, the primary task is to build a language model that correctly predicts the center word given the context words in which the center word appears.

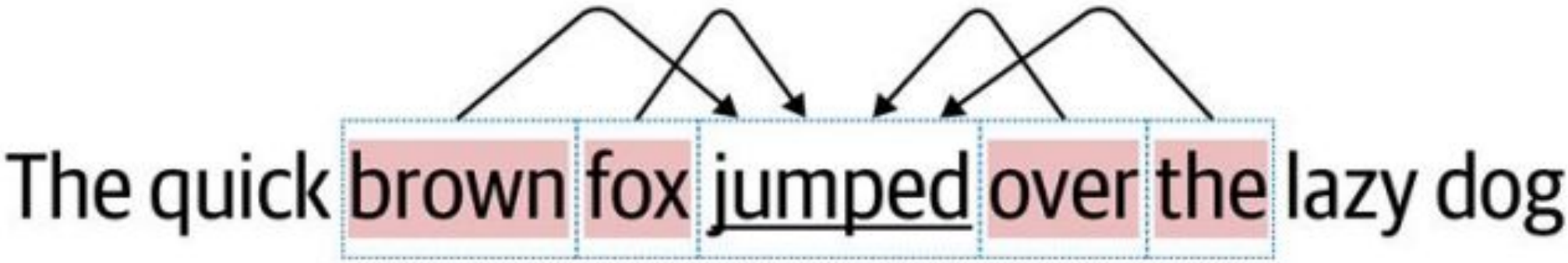
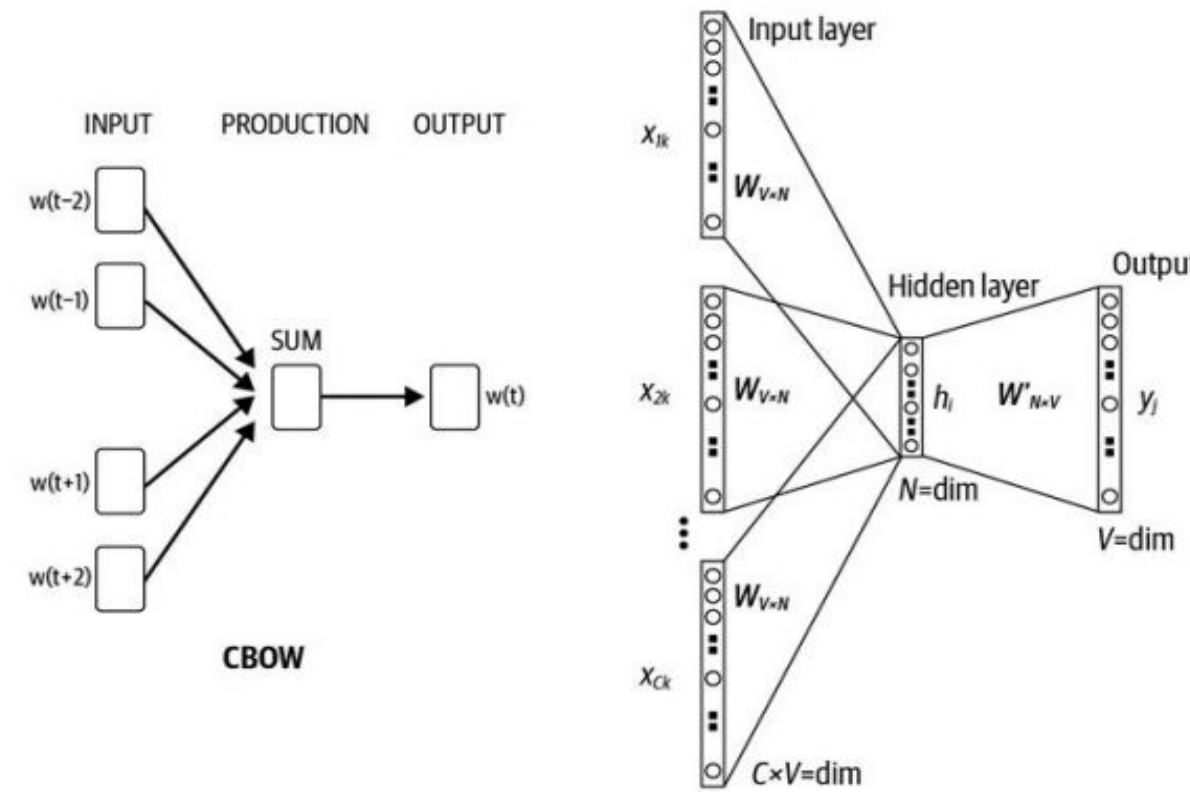
- CBOW tries to learn a language model that tries to predict the “center” word from the words in its context



CBOW Model



CBOW Model



Source Text

Training Samples
(context, target)

- The quick brown fox jumps over the lazy dog. → ((quick, brown), The)
- The quick brown fox jumps over the lazy dog. → ((The, brown, fox), quick)
- The quick brown fox jumps over the lazy dog. → ((The, quick, fox, jumps), brown)
- The quick brown fox jumps over the lazy dog. → ((quick, brown, jumps, over), fox)

```
import sys
!{sys.executable} -m pip install gensim

from gensim.models import Word2Vec

# Sample data
sentences = [
    ['the', 'Quick', 'brown', 'fox', 'jumped', 'over', 'the', 'lazy', 'dog']]

# Initialize the Word2Vec model
model = Word2Vec(sentences, vector_size=25, window=5, min_count=1, workers=4)

# Train the model
model.train(sentences, total_examples=len(sentences), epochs=10)

print(model.wv['dog'])
```



```
import sys
!{sys.executable} -m pip install gensim

from gensim.models import Word2Vec

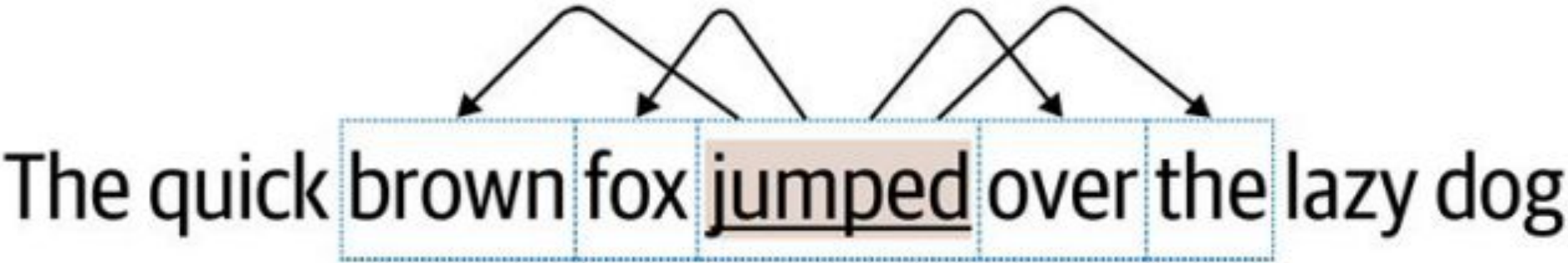
# Sample data
sentences = [
    ['this', 'is', 'the', 'first', 'document'],
    ['this', 'document', 'is', 'the', 'second', 'document'],
    ['and', 'this', 'is', 'the', 'third', 'one'],
    ['is', 'this', 'the', 'first', 'document']
]

# Initialize the Word2Vec model
model = Word2Vec(sentences, vector_size=20, window=5, min_count=1, workers=4)

# Train the model
model.train(sentences, total_examples=len(sentences), epochs=10)

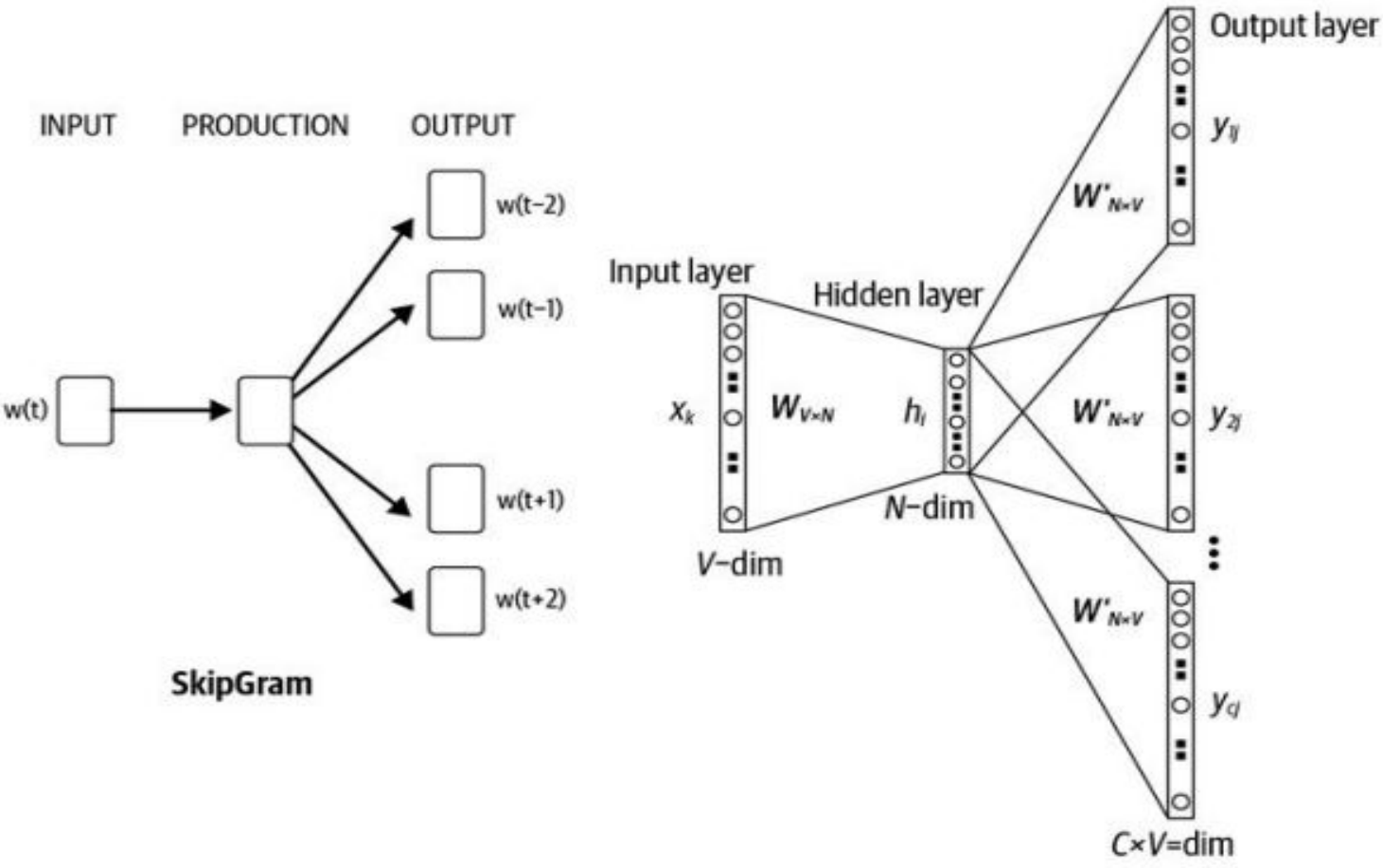
# Get vector for a word
print(model.wv['document'])
```

- Skip Gram is very similar to CBOW, with some minor changes.
- In Skip Gram, the task is to predict the context words from the center word

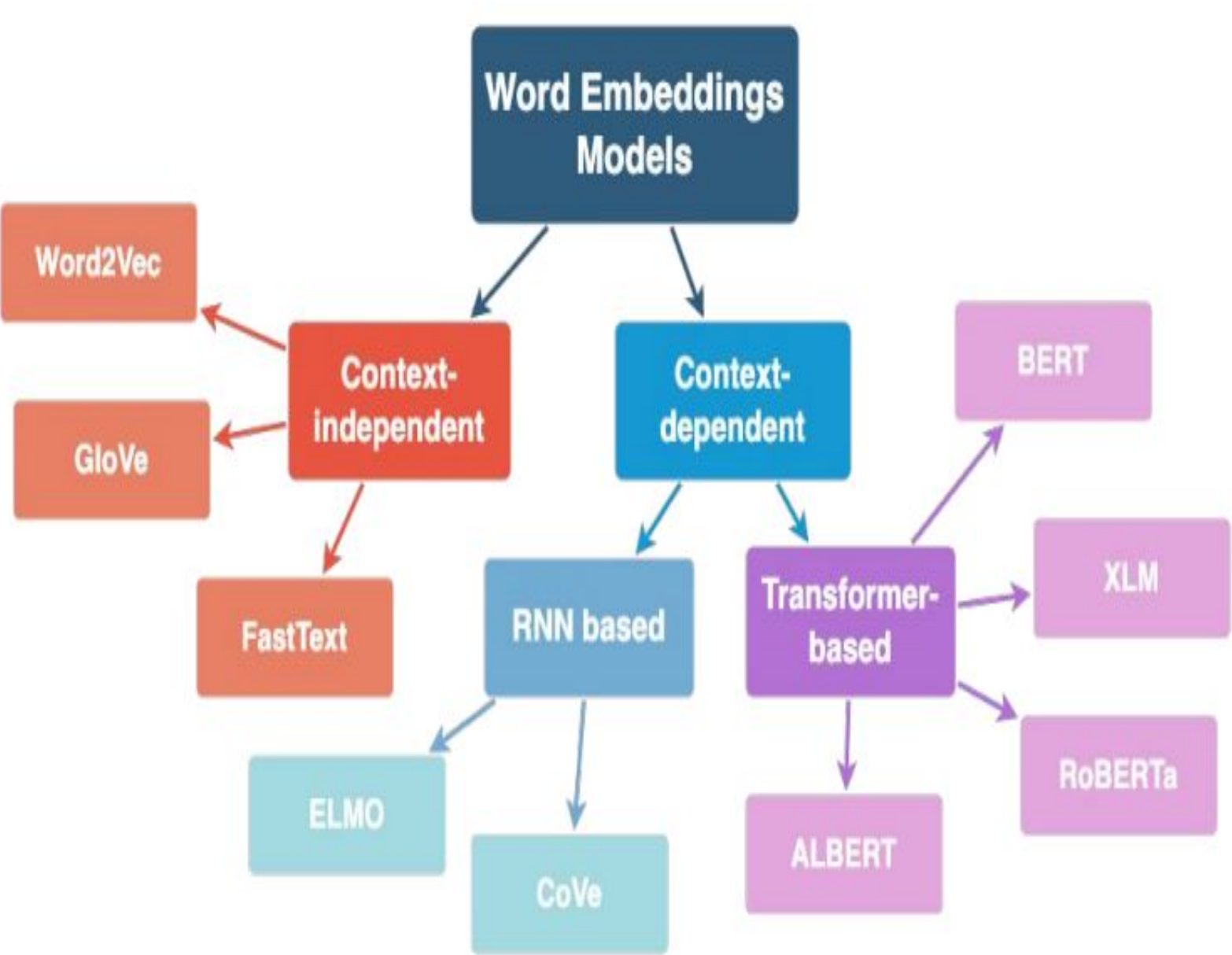
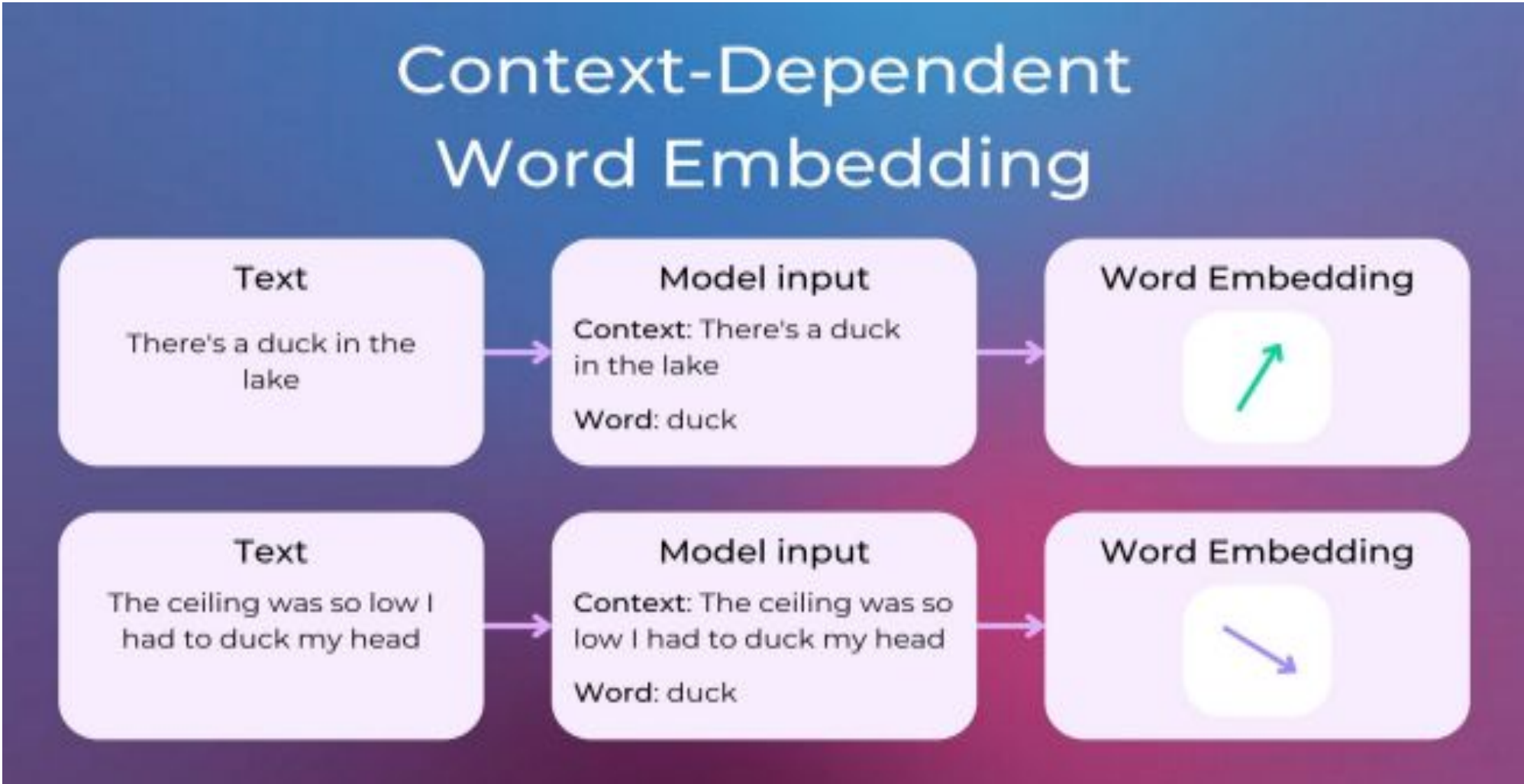


Continuous SkipGram Model

Source Text	Training Samples (target, context)
The quick brown fox jumps over the lazy dog.	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	(fox, quick) (fox, brown) (fox, jumps) (fox, over)



Context-dependent Embeddings



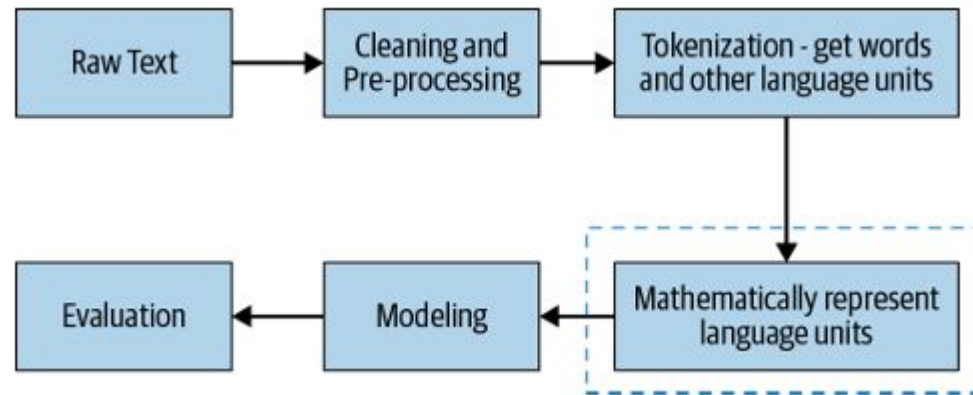
Text Representation

In language processing, the vectors x are derived from textual data, in order to reflect various linguistic properties of the text.

—Yoav Goldberg

Text Representation

- To transform a given text into numerical form so that it can be fed into NLP and ML algorithms
- conversion of raw text to a suitable numerical (numerical vector) form is called **text representation**



Feature representation: Image

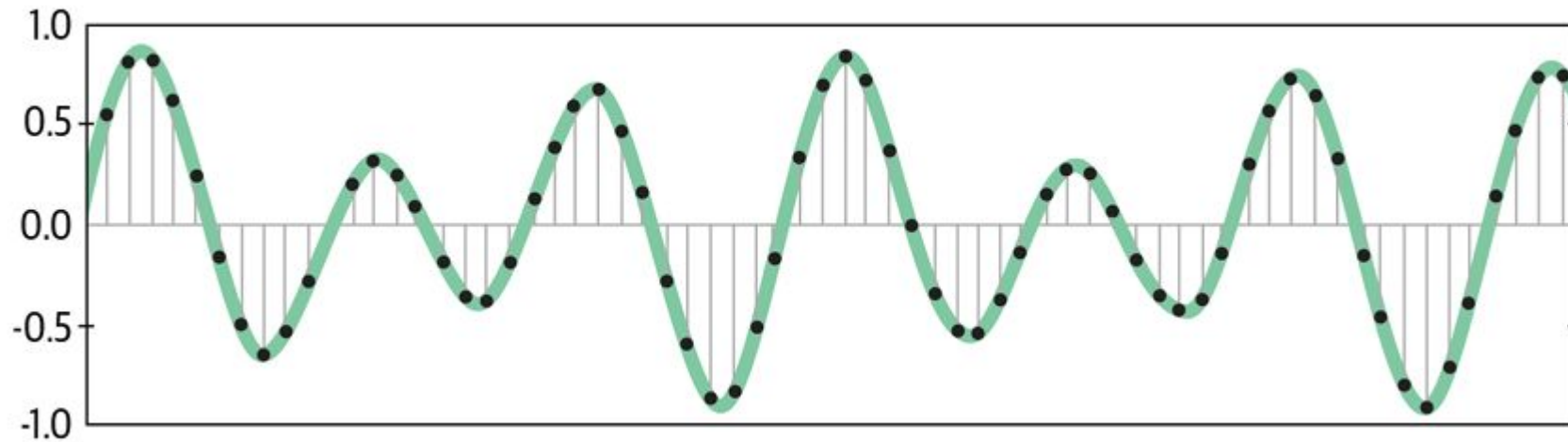


What We See

08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	08
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	48	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	53	88	30	03	49	13	36	65
52	70	95	23	04	60	11	42	69	24	68	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	63	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	32	60	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
88	36	68	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	69
04	42	16	73	38	25	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	62	99	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	86	81	16	23	57	05	34
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	19	67	48

What Computers See

Feature representation: Speech



```
[-1274, -1252, -1160, -986, -792, -692, -614, -429, -286, -134, -57, -41,  
-169, -456, -450, -541, -761, -1067, -1231, -1047, -952, -645, -489, -448,  
-397, -212, 193, 114, -17, -110, 128, 261, 198, 390, 461, 772, 948, 1451,  
1974, 2624, 3793, 4968, 5939, 6057, 6581, 7302, 7640, 7223, 6119, 5461,  
4820, 4353, 3611, 2740, 2004, 1349, 1178, 1085, 901, 301, -262, -499,  
-488, -707, -1406, -1997, -2377, -2494, -2605, -2675, -2627, -2500, -2148,  
-1648, -970, -364, 13, 260, 494, 788, 1011, 938, 717, 507, 323, 324, 325,  
350, 103, -113, 64, 176, 93, -249, -461, -606, -909, -1159, -1307, -1544]
```


State-of-the-art techniques for representing text

- ⊖ Basic vectorization approaches
- ⊖ Distributed representations
- ⊖ Universal language representation
- ⊖ Handcrafted features

Vector Space Models

- The text data must be converted into some mathematical form
- represent **text units** (characters, phonemes, words, phrases, sentences, paragraphs, and documents) with **vectors of numbers**
- This is known as the **vector space model (VSM)**
- VSM is a simple algebraic model used extensively for representing any text blob
- **Applications:** information-retrieval operations, document classification and document clustering

- It's a mathematical model that represents **text units as vectors**
- these are **vectors of identifiers**, such as index numbers in a corpus vocabulary

Familiar with Terminologies

- **Document**

A document is a single text data point.

For Example, a review of a particular product by the user.

- **Corpus**

It a collection of all the documents present in our dataset.

- **Feature**

Every unique word in the corpus is considered as a feature.

- **vocabulary**

set of all unique words (tokens) that appear in a given corpus

Vectorization Approaches for Text Representation

- Representing text data in a numerical format is a fundamental step in many natural language processing (NLP) tasks
- One of the basic approaches is to **map each unique word** in the vocabulary **to a unique integer ID**, and then represent each document as a **vector of these IDs**.
- This simple yet powerful technique, known as **one-hot encoding**, forms the foundation for more advanced text vectorization methods

Example

The toy corpus with 4 documents:

- - D1: Dog bites man.
- - D2: Man bites dog.
- - D3: Dog eats meat.
- - D4: Man eats food.

Vocabulary from the toy corpus:

[dog, bites, man, eats, meat, food]

One-Hot Encoding

1

Intuitive Representation

One-hot encoding is a simple and straightforward way to represent words, where each unique word in the vocabulary is assigned a unique integer ID and represented by a binary vector with a single 1 at the corresponding index.

2

Sparsity and Inefficiency

The size of a one-hot vector is directly proportional to the size of the vocabulary, leading to high-dimensional and sparse representations that are computationally inefficient to store, compute, and learn from.

3

Lack of Semantic Relationships

One-hot encoding treats words as atomic units and does not capture any notion of similarity or relatedness between words, which is crucial for many NLP tasks.

4

Out-of-Vocabulary Problem

One-hot encoding cannot handle new words that were not present in the training data, leading to the out-of-vocabulary (OOV) problem and the need to retrain the model.

Implementing One-Hot Encoding

Vocabulary Mapping

The first step in one-hot encoding is to **create a mapping between unique words in the corpus and their corresponding integer IDs**. This can be done by iterating through the corpus and building a dictionary that maps each word to a unique ID.

Vector Representation

Once the vocabulary mapping is established, each word can be **represented as a binary vector of the same length as the vocabulary**, with a single 1 at the index corresponding to the word's ID and 0s elsewhere.

Document Representation

To represent a document, the one-hot vectors of its constituent words are simply **concatenated to form a matrix**, where each row corresponds to a word and the columns represent the vocabulary.

Vocabulary Mapping

- Each word is assigned a unique ID:
 - dog = 1
 - bites = 2
 - man = 3
 - meat = 4
 - food = 5
 - eats = 6

Vector Representation

dog = 1

VR: [1 0 0 0 0 0]

bites = 2

VR: [0 1 0 0 0 0]

meat = 4

VR: ?

Document Representation

- Example of document D1 representation:

- D1: Dog bites man

DR: [[1 0 0 0 0 0] [0 1 0 0 0 0] [0 0 1 0 0 0]]

Python Implementation

```
def get_onehot_vector(somestring):  
    onehot_encoded = []  
    for word in somestring.split():  
        temp = [0]*len(vocab)  
        if word in vocab:  
            temp[vocab[word]-1] = 1  
        onehot_encoded.append(temp)  
    return onehot_encoded
```

```
get_onehot_vector(processed_docs[1])
```

```
Output: [[0, 0, 1, 0, 0, 0], [0, 1, 0, 0, 0, 0], [1, 0, 0, 0, 0, 0]]
```

Limitations of One-Hot Encoding

1

Sparsity

One-hot encoded vectors are inherently sparse, with most of the elements being zero. This sparsity can lead to computational inefficiencies and make it difficult for machine learning models to learn meaningful patterns from the data.

2

Fixed-Length Representation

One-hot encoding does not provide a fixed-length representation for text, as the length of the vector depends on the number of unique words in the corpus. This can be problematic for many machine learning algorithms that require inputs of a consistent size.

3

Lack of Semantic Relationships

One-hot encoding treats words as completely independent entities, failing to capture any semantic relationships or similarities between them. This can limit the performance of models in tasks that rely on understanding the meaning and context of words.

Bag of Words

- Bag of words (BoW) is a classical text representation technique that has been used commonly in NLP, especially in text classification problems
- It involves converting a text into a set of words (or tokens) and then **representing the text as a vector of word frequencies** or binary indicators.

- The key idea behind it is as follows: represent the text under consideration as a bag (collection) of words while ignoring the order and context.
- The basic intuition behind it is that it assumes that **the text belonging to a given class in the dataset is characterized by a unique set of words.**
- If two text pieces have nearly the same words, then they belong to the same bag (class).
- Thus, by analyzing the words present in a piece of text, one can identify the class (bag) it belongs to.

- BoW maps words to unique integer IDs between 1 and $|V|$.
- Each document in the corpus is then converted into a vector of $|V|$ dimensions where in the i th component of the vector, $i = w_{id}$, is simply the number of times the word 'w' occurs in the document,
- i.e., we simply score each word in V by their **occurrence count** in the document.


```
S1 := "i am doing great as i was before"  
S2 := "this is great"  
S3 := "playing sport is a good habit"
```

taking all words and their occurrences into consideration, we get:

	i	am	doing	great	as	was	before	this	good	used	to	playing	sport	is	a	habit
"i am doing great as i was before"	2	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
"this is great"	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0
"playing sport is a good habit"	0	0	0	0	0	0	0	0	1	0	0	1	1	1	1	1

Bag of words

Bag of N-Grams

- combining consecutive 2 words we could extract much more information from the documents.
- E.G. “GOOD” and “VERY GOOD”
- This is known as **Bi-Grams**

	i am	am doing	doing great	great as	as i	i was	was before	this is	is great	playing sport	sport is	is a	a good	good habit
“i am doing great as i was before”	1	1	1	1	1	1	1	0	0	0	0	0	0	0
“this is great”	0	0	0	0	0	0	0	1	1	0	0	0	0	0
“playing sport is a good habit”	0	0	0	0	0	0	0	0	0	1	1	1	1	1

2-Grams approach

- We can extend this to n-words meaning “**N-Grams**” approach (Natural extension of BOW model).
- So we will have a sentence mapped to features (columns as mentioned in diagram) and the number of features representing this will be $L-N+1$, L being number of tokens and N being N-grams.

Training N-grams:

Predicting probability of sequence of words: ($w = (w_1 w_2 w_3 w_4 w_5 \dots w_k)$)

$$P(w) = P(w_1).P(w_2|w_1).P(w_3|w_2,w_1)\dots P(w_k | w_1, w_2, \dots, w_{k-1})$$

Markov Assumption:

Instead of considering all the words let's just take top n words, equation changed to:

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_i | w_{i-n+1}, \dots, w_{i-1})$$

Bi-Gram language model:

Since we are dealing with pairs, we can say Probability of next word occurring given previous word occurred.

$$P(w) = P(w_1).P(w_2 | w_1).P(w_3 | w_2).... P(w_k | w_{k-1})$$

Example: Bi-gram Probability calculation

```
S1 := "This is a bar"  
S2 := "This is not a challenge"  
S3 := "Assume this was a challenge"
```

- $P(\text{this is}) = P(\text{this}) \cdot P(\text{is} \mid \text{this})$
 $= (3/14) \cdot (2/3)$
 $= 2/21$

$P(\text{This is a challenge}) = ?$

TF-IDF

- TF-IDF, **Term frequency–inverse document frequency**
- In previous methods, All the words in the text are treated as equally important—there's no notion of some words in the document being more important than others
- TF-IDF addresses this issue
- It aims to quantify the **importance of a given word relative to other words** in the document and in the corpus.
- It's a commonly used representation scheme for information-retrieval systems, for extracting relevant documents from a corpus for a given text query

The intuition behind TF-IDF is as follows:

- if a word 'w' appears many times in a document 'd_i' but does not occur much in the rest of the documents 'd_j' in the corpus, then the word 'w' must be of great importance to the document 'd_i'.
- The importance of 'w' **should increase in proportion to its frequency** in 'd_i', but at the same time, its importance should decrease in proportion to the word's frequency in other documents 'd_j' in the corpus.
- Mathematically, this is captured using two quantities: **TF and IDF**.
- The two are then combined to arrive at the TF-IDF score.

Corpus Example

- **Article 1:** "Breaking news: The stock market crashed today due to unexpected economic data."
- **Article 2:** "Sports update: The local football team won their game with a last-minute goal."
- **Article 3:** "Technology: New smartphone model released with innovative features and improved battery life."

TF (term frequency)

- TF (term frequency) measures **how often a term or word occurs** in a given document.
- Since different documents in the corpus may be of different lengths, a term may occur more often in a longer document as compared to a shorter document.
- To normalize these counts, we divide the number of occurrences by the length of the document.
- TF of a term t in a document d is defined as:

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

IDF (inverse document frequency)

- IDF (inverse document frequency) measures the **importance of the term across a corpus**.
- In computing TF, all terms are given equal importance (weightage).
- However, it's a well-known fact that stop words like is, are, am, etc., are not important, even though they occur frequently.
- To account for such cases, IDF **weighs down** the **terms that are very common across a corpus** and **weighs up** the **rare terms**.
- IDF of a term 't' is calculated as follows:

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

TF-IDF score

- The TF-IDF score is a product of these two terms.
- Thus, **TF-IDF score = TF * IDF**.

- Let's compute TF-IDF scores for our toy corpus.

- - D1: Dog bites man.
- - D2: Man bites dog.
- - D3: Dog eats meat.
- - D4: Man eats food.

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

$$\text{IDF}(t) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

- Some terms appear in only one document, some appear in two, while others appear in three documents.
- The size of our corpus is $N=4$.

Word	TF score	IDF score	TF-IDF score
dog	$\frac{1}{3} = 0.33$	$\log_2(4/3) = 0.4114$	$0.4114 * 0.33 = 0.136$
bites	$\frac{1}{6} = 0.17$	$\log_2(4/2) = 1$	$1 * 0.17 = 0.17$
man	0.33	$\log_2(4/3) = 0.4114$	$0.4114 * 0.33 = 0.136$
eats	0.17	$\log_2(4/2) = 1$	$1 * 0.17 = 0.17$
meat	$\frac{1}{12} = 0.083$	$\log_2(4/1) = 2$	$2 * 0.083 = 0.17$
food	0.083	$\log_2(4/1) = 2$	$2 * 0.083 = 0.17$

The TF-IDF vector representation for a document is then simply the TF-IDF score for each term in that document. So, for D_1 we get

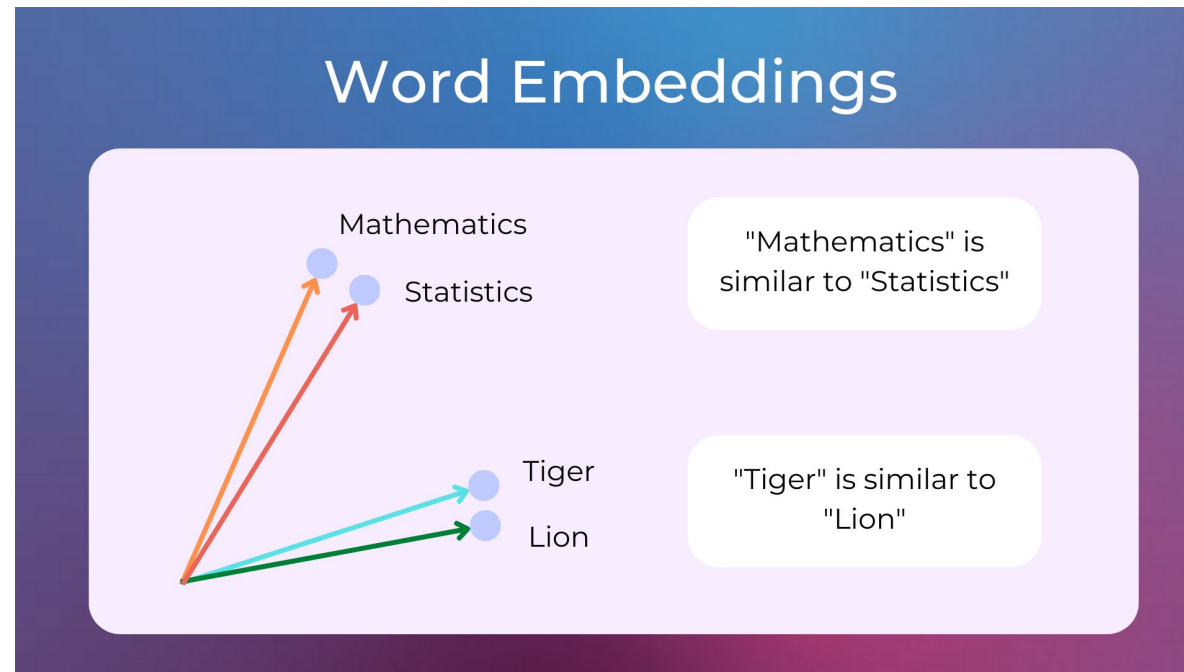
Dog	bites	man	eats	meat	food
0.136	0.17	0.136	0	0	0

- Similar to BoW, we can use the TF-IDF vectors to calculate similarity between two texts using a similarity measure like Euclidean distance or cosine similarity.
- TF-IDF is a commonly used representation in application scenarios such as information retrieval and text classification.
- However, despite the fact that TF-IDF is better than the vectorization methods we saw earlier in terms of capturing similarities between words, it still suffers from the curse of high dimensionality.

Representing Texts as Vectors: Word Embeddings

- TF-IDF works quite well for several use cases.
- A downside of TF-IDF (and of bag of words approaches in general) is that the semantics of the tokens is not taken into account.
- For example, the tokens “soccer” and “football” should have similar representations as they are both about sports, while their representations should be very different from that of a word like “red”.
- It would be great to represent tokens in a way so that semantically similar tokens have similar representations: we can do that with **word embeddings**.

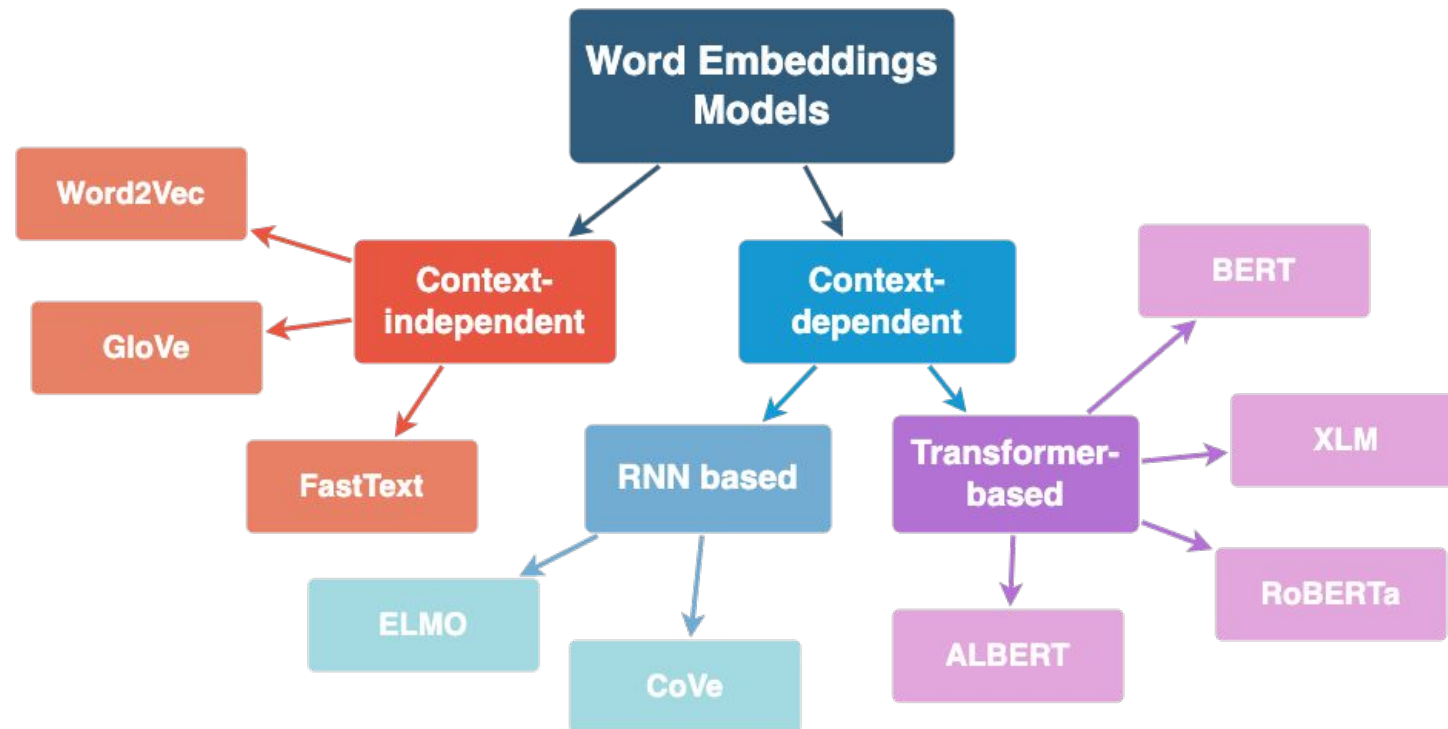
- Word embeddings are a type of word representation (with numerical vectors) that allows words with similar meanings to have a similar representation.



Why Use Word Embeddings?

- **Semantic Similarity:** Embeddings allow for capturing semantic relationships between words. For example, the words "dog," "puppy," and "cat" might be close to each other in the vector space.
- **Dimensionality Reduction:** Unlike one-hot encoding, which results in sparse and high-dimensional vectors, embeddings provide a dense, lower-dimensional representation.
- **Transfer Learning:** Pre-trained embeddings can be used in various NLP tasks without training from scratch, saving computational resources and improving performance.

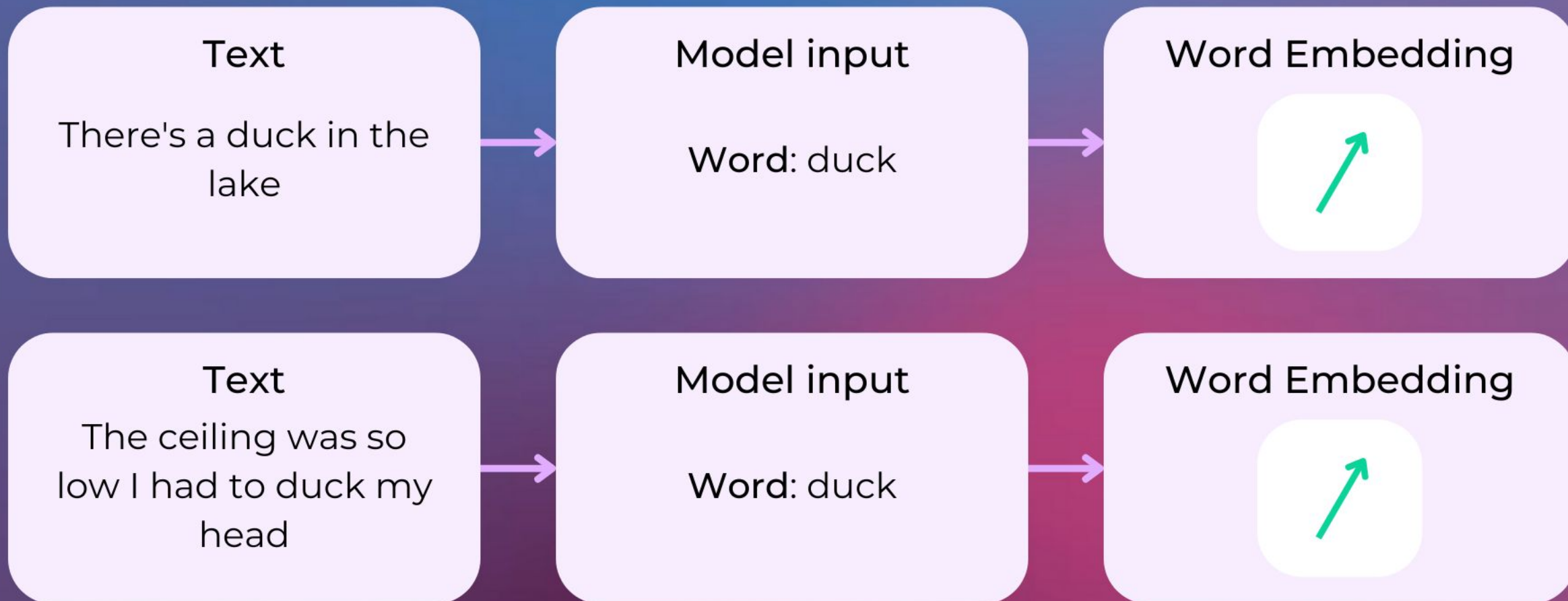
word embedding methods



Context-independent Embeddings

- With context-independent embeddings models, the learned representations are characterized by **being unique and distinct for each word, without considering the word's context.**
- This means that homonyms like “duck” (which can be the animal or a verb) won't be disambiguated by their context (i.e. the text in which the word appears) and will always have a unique word embedding that somehow captures both meanings.
- As a result, these models **produce a map between all the words in the vocabulary and their vectors.**

Context-Independent Word Embedding



Popular Word Embedding Models

Context-independent
Pre-Trained

- **Word2Vec:**

- Developed by Google.
- Two main models: **CBOW** (Continuous Bag of Words) and **Skip-gram**.
 - CBOW (Continuous Bag of Words): Predicts the current word based on the context (surrounding words).
 - Skip-gram: Predicts the surrounding words given the current word.
- Learns word vectors by predicting a word from its context (CBOW) or the context from a word (Skip-gram).
- Example: In Word2Vec, the word "king" might be close to "queen," "royal," and "monarch" in the vector space, indicating similar meanings or associations.

Popular Word Embedding Models

Context-independent

Pre-Trained

- **GloVe (Global Vectors for Word Representation):**
 - Developed by Stanford.
 - Uses global word-word co-occurrence statistics from a corpus to learn embeddings.
 - Captures both global statistical information and local context.

Popular Word Embedding Models

Context-independent

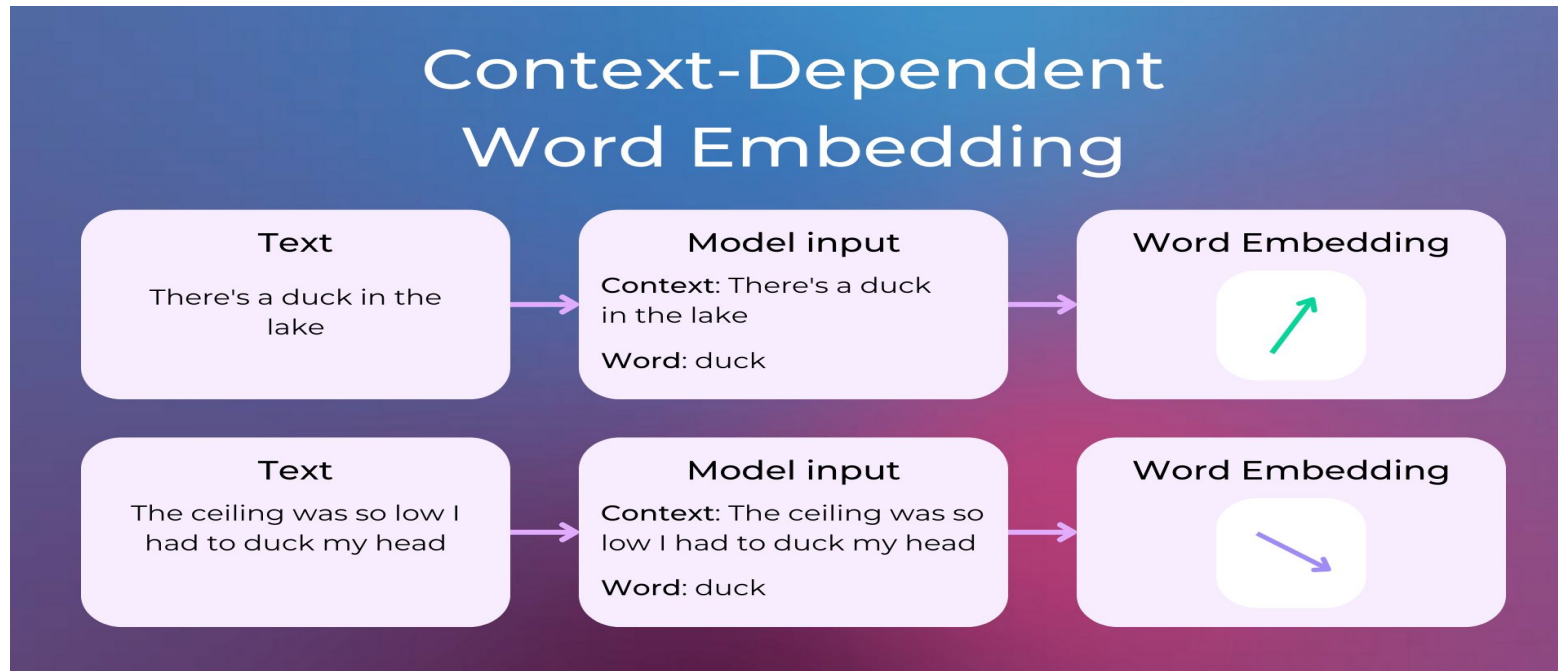
Pre-Trained

- **fastText:**

- Developed by Facebook's AI Research (FAIR) lab.
- Extends Word2Vec by considering subword information (character n-grams).
- Can handle out-of-vocabulary (OOV) words better by understanding the morphology of words.

Context-dependent Embeddings

- Unlike context-independent word embeddings, context-dependent methods learn different embeddings for the same word **based on its context.**



Context-dependent and RNN based

- **ELMO (Embeddings from Language Model)**: Learns contextualized word representations based on a neural language model with a character-based encoding layer and two BiLSTM layers.
- **CoVe (Contextualized Word Vectors)**: Uses a deep LSTM encoder from an attentional sequence-to-sequence model trained for machine translation to contextualize word vectors.

Context-dependent and Transformer-based

- **BERT (Bidirectional Encoder Representations from Transformers)**: Transformer-based language representation model trained on a large cross-domain corpus. Applies a masked language model to predict words that are randomly masked in a sequence, and this is followed by a next-sentence-prediction task for learning the associations between sentences.
- **XLM (Cross-lingual Language Model)**: It's a transformer pre-trained using next token prediction, a BERT-like masked language modeling objective, and a translation objective.
- **RoBERTa (Robustly Optimized BERT Pretraining Approach)**: It builds on BERT and modifies key hyperparameters, removing the next-sentence pretraining objective and training with much larger mini-batches and learning rates.
- **ALBERT (A Lite BERT for Self-supervised Learning of Language Representations)**: It presents parameter-reduction techniques to lower memory consumption and increase the training speed of BERT.

Training our own embeddings

- Two architectural variants that were proposed in the original Word2vec approach.

The two variants are:

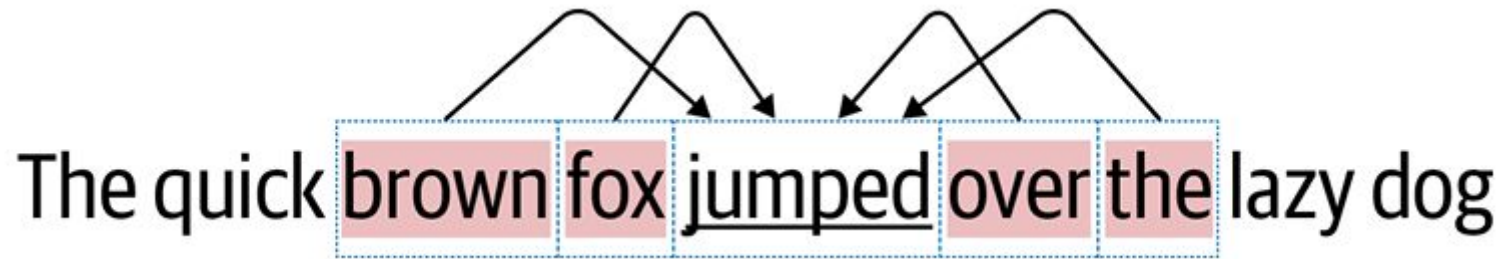
- **Continuous bag of words (CBOW)**
- **SkipGram**

Continuous bag of words (CBOW)

- In CBOW, the primary task is to build a language model that correctly **predicts the center word** given the context words in which the center word appears.
- CBOW tries to learn a language model that tries to predict the “center” word from the words in its context
- What is a language model?
- It is a (statistical) model that tries to give a probability distribution over sequences of words.
- Given a sentence of, say, m words, it assigns a probability $\Pr(w_1, w_2, \dots, w_n)$ to the whole sentence.

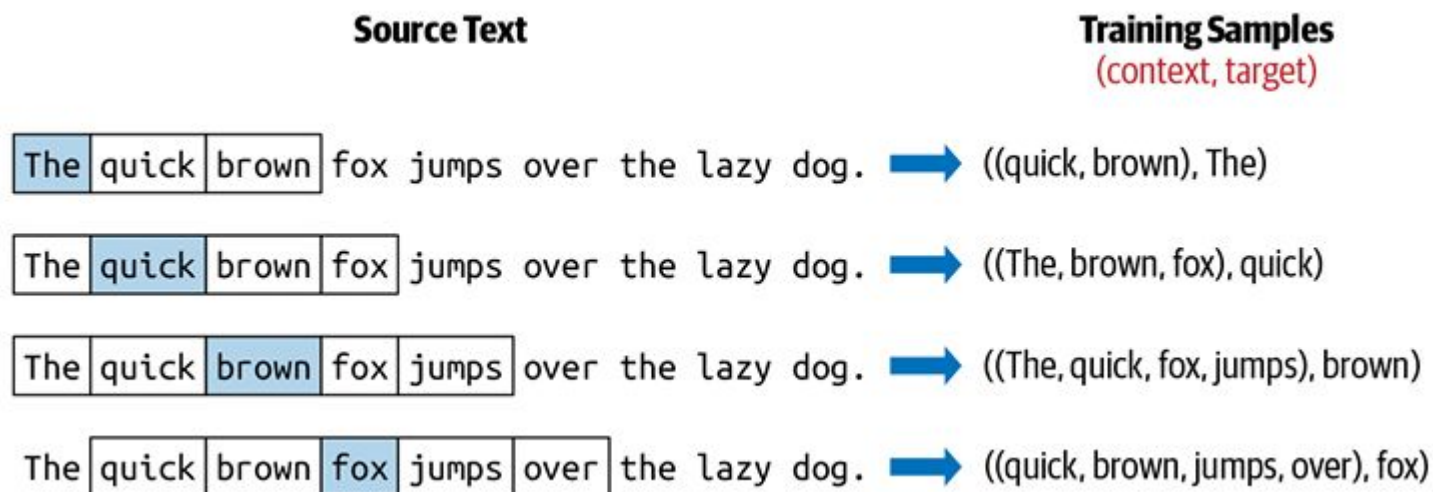
- The objective of a language model is to assign probabilities in such a way that it gives high probability to “good” sentences and low probabilities to “bad” sentences.
- By good, we mean sentences that are **semantically and syntactically correct**.
- By bad, we mean **sentences that are incorrect**—semantically or syntactically or both.
- So, for a sentence like “The cat jumped over the dog,” it will try to assign a probability close to 1.0, whereas for a sentence like “jumped over the cat dog,” it tries to assign a probability close to 0.0.

Example:



- run a sliding window of size $2k+1$ over the text corpus.
- we took k as 2.
- Each position of the window marks the set of $2k+1$ words that are under consideration.
- The center word in the window is the target, and k words on either side of the center word form the context.
- This gives us one data point.

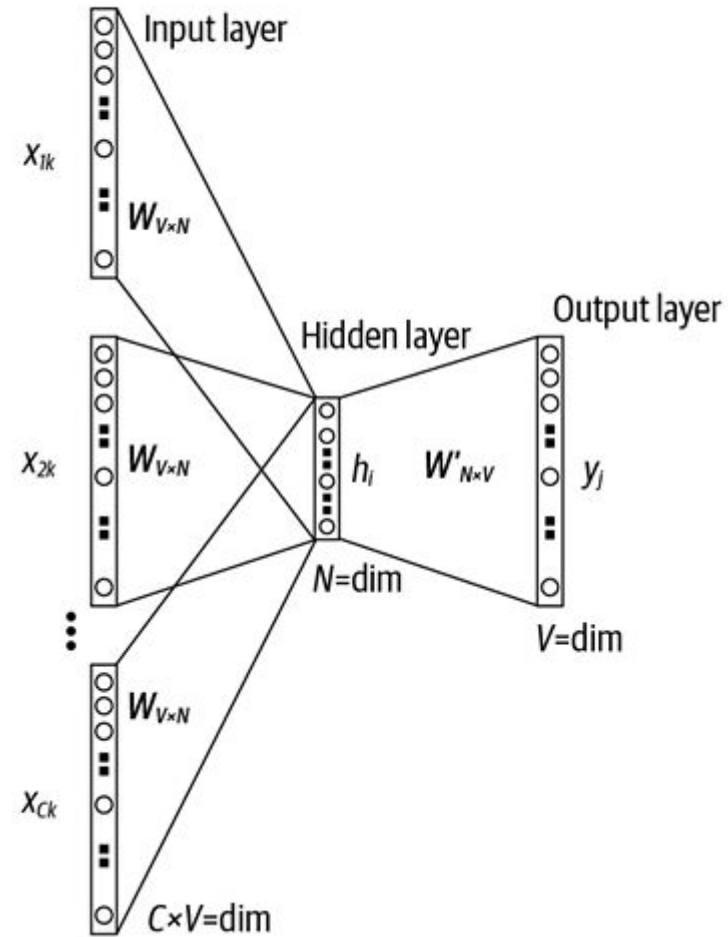
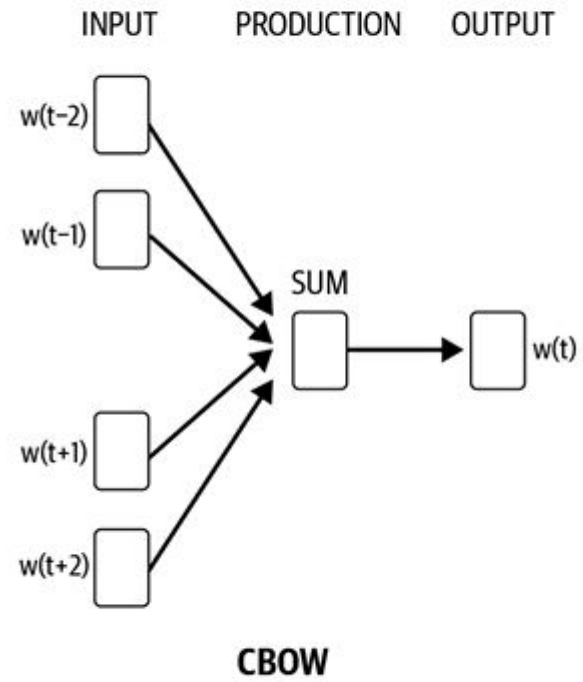
- A single data point consists of a pair of numbers: (2k indices of words in context, index of word in target).
- To get the next data point, simply shift the window to the right on the corpus by one word and repeat the process.
- This way, slide the window across the entire corpus to create the training set.



CBOW model

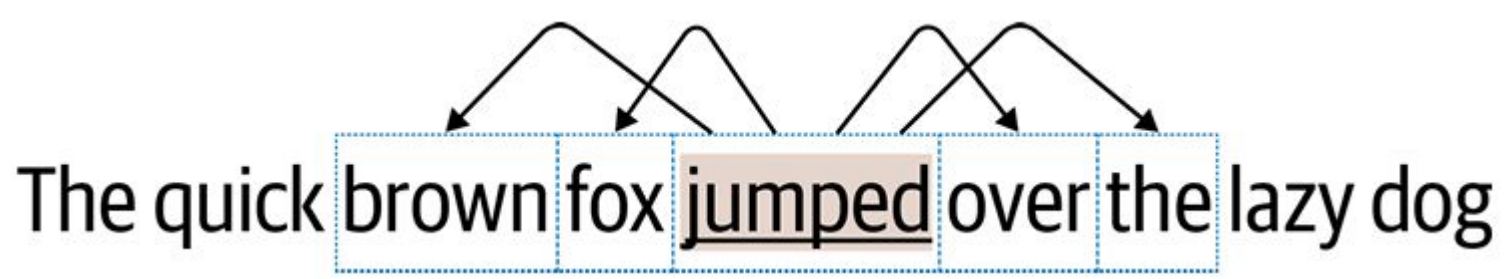
- The objective is to learn an embedding matrix $E \in \mathbb{R}^{|V| \times d}$.
- To begin with, we initialize the matrix randomly.
- Here, $|V|$ is the size of corpus vocabulary and d is the dimension of the embedding.
- A simple ANN (fully connected) is used to implement CBOW
- At the end of the training, E is the embedding matrix we wanted to learn

CBOW Model



SkipGram

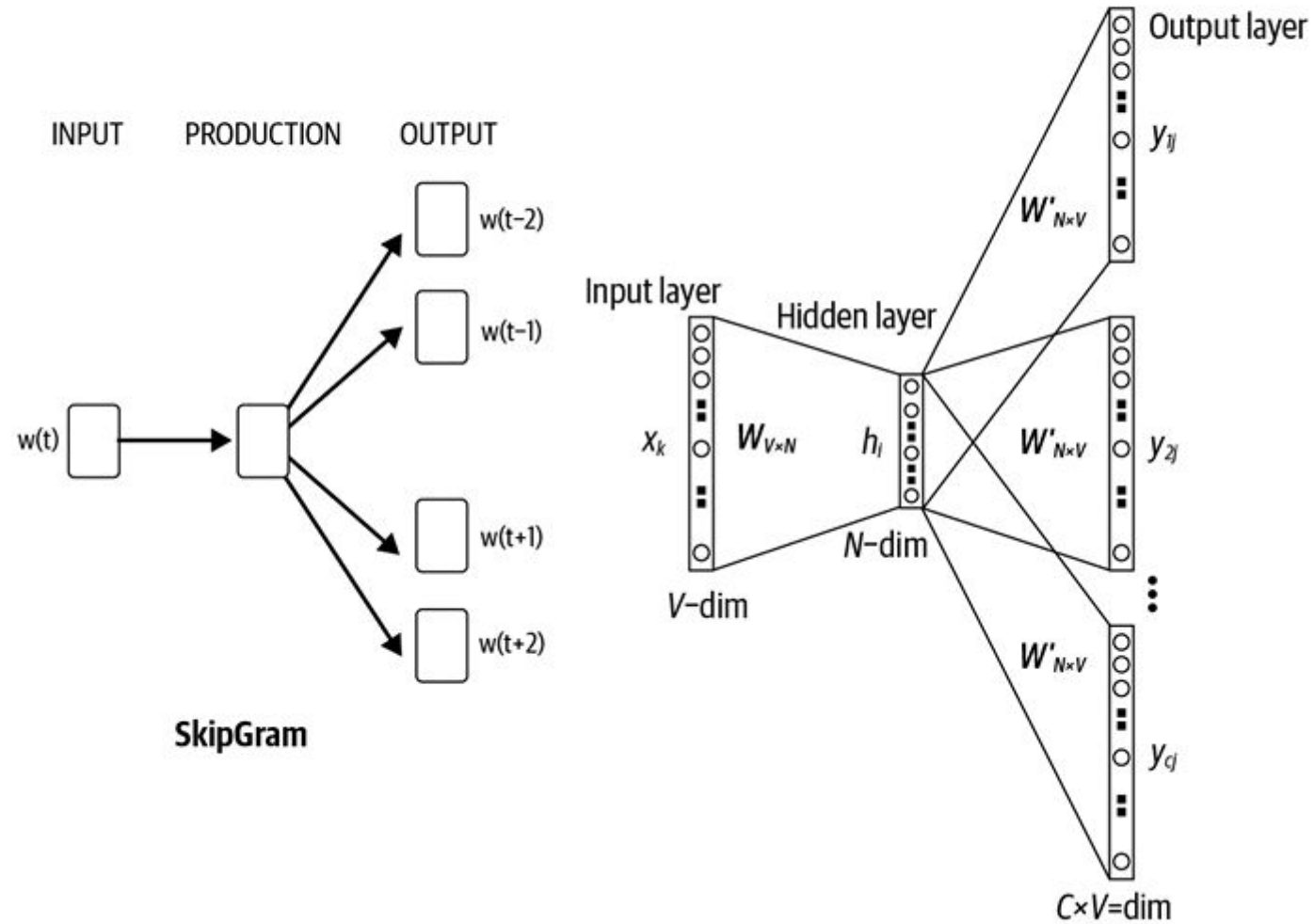
- SkipGram is very similar to CBOW, with some minor changes.
- In Skip Gram, the task is to predict the context words **from the center word**.



Training in skip gram

Source Text	Training Samples (target, context)					
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)		
The	quick	brown				
The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
quick	brown	fox				
The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)		
brown	fox	jumps				
The <table><tr><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
quick	brown	fox	jumps	over		

Continuous SkipGram Model



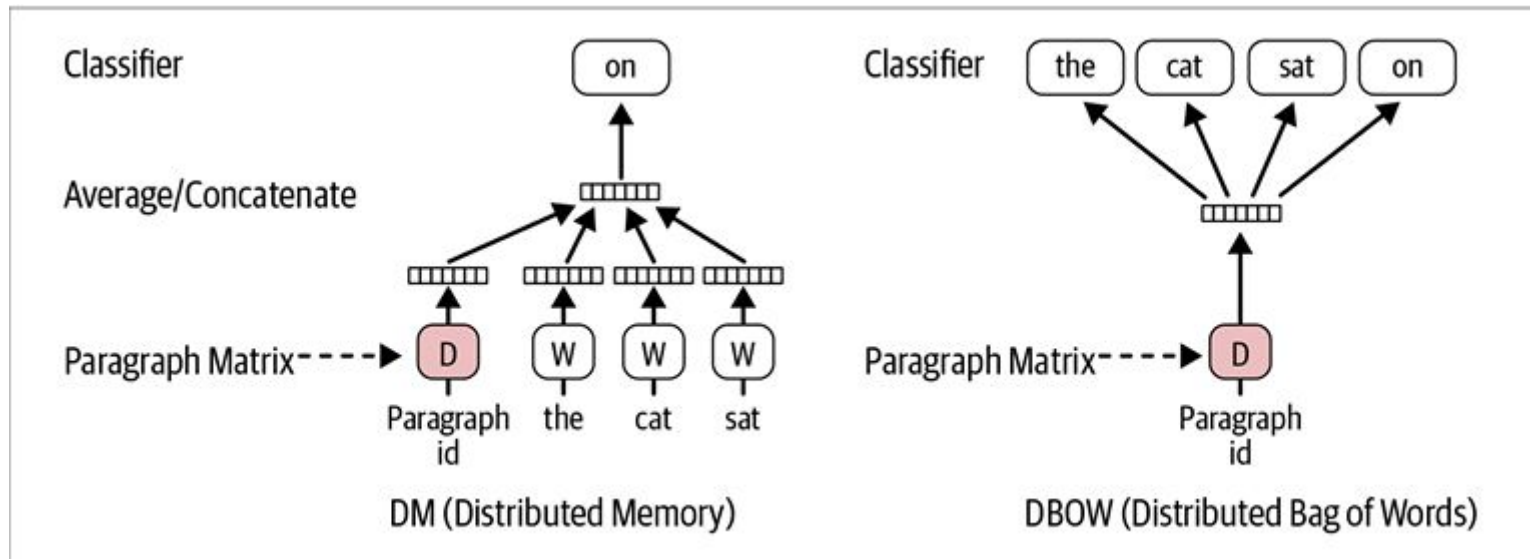
Distributed Representations Beyond Words and Characters

- Word2vec learned representations for words, and we aggregated them to form text representations.
- fastText learned representations for character n-grams, which were aggregated to form word representations and then text representations.
- A potential problem with both approaches is that they do not take the context of words into account.
- *“dog bites man” and “man bites dog.” Both receive the same representation in these approaches, but they obviously have very different meanings*
- *Solution: Doc2Vec*

Doc2vec

- Allows us to directly learn the representations for texts of **arbitrary lengths** (phrases, sentences, paragraphs, and documents) by taking the context of words in the text into account.
- Doc2vec is based on the paragraph vectors framework
- This is similar to Word2vec in terms of its general architecture, except that, in addition to the word vectors, it also learns a “**paragraph vector**” that learns a representation for the full text (i.e., with words in context)

- The shallow neural networks used to learn Doc2vec embeddings are very similar to the CBOW and SkipGram architecture of Word2vec.
- The two architectures are called **distributed memory (DM)** and **distributed bag of words (DBOW)**.



Text classification

- Text classification is a Natural Language Processing task where a piece of text is assigned to **one or more categories**.
- In the email spam–identifier example, we have two categories—spam and non-spam— and each incoming email is assigned to one of these categories.
- This can be achieved through various approaches, including rule-based methods, machine learning algorithms, and deep learning techniques

Rule-Based Methods

- These rely on manually created rules to classify text.
- While simple and interpretable, they are less flexible and can be difficult to scale.

Machine Learning Algorithms

- **Naive Bayes:** A probabilistic classifier based on Bayes' theorem, suitable for text classification due to its simplicity and effectiveness.
- **Support Vector Machines (SVM):** These find the hyperplane that best separates different classes in the feature space.
- **Decision Trees and Random Forests:** These create a model that predicts the value of a target variable based on several input variables.

Deep Learning Techniques

- **Recurrent Neural Networks (RNNs):** Particularly useful for sequential data like text. Long Short-Term Memory (LSTM) networks are a type of RNN designed to remember long-term dependencies.
- **Convolutional Neural Networks (CNNs):** Though traditionally used for image data, they can be effective for text classification, especially for capturing local dependencies.
- **Transformers:** These models, like BERT, GPT, and their variants, have set new standards in NLP tasks, including text classification, due to their ability to handle long-range dependencies and contextual information.

Applications of Text Classification

Overview of Various Uses in Different Fields

Sentiment Analysis

- Objective: Determine the sentiment (positive, negative, neutral) expressed in a piece of text.
- Applications:
 - - Analyzing customer feedback on products and services.
 - - Monitoring social media sentiment for brands or public figures.
 - - Gauging public opinion on political issues.

Spam Detection

- Objective: Identify and filter out spam emails or messages.
- Applications:
 - - Email filtering systems.
 - - Social media platforms detecting spammy posts or comments.
 - - SMS spam filtering.

Document Categorization

- Objective: Classify documents into predefined categories.
- Applications:
 - - Organizing news articles by topics (e.g., sports, politics, technology).
 - - Tagging academic papers by research area.
 - - Sorting customer service tickets into different issue types.

Language Detection

- Objective: Identify the language of a given text.
- Applications:
 - - Enabling multilingual support in software applications.
 - - Preprocessing step for other NLP tasks in multilingual settings.
 - - Filtering and routing content based on language.

Topic Labeling

- Objective: Assign topics to a piece of text.
- Applications:
 - - Summarizing large volumes of text data.
 - - Enhancing search engine algorithms.
 - - Content recommendation systems.

Intent Detection

- Objective: Identify the user's intent in a text.
- Applications:
 - - Virtual assistants and chatbots understanding user queries.
 - - Customer support systems routing queries to the appropriate department.
 - - E-commerce platforms interpreting user search queries.

Named Entity Recognition (NER)

- Objective: Classify named entities in text into predefined categories (e.g., person names, organizations, locations).
- Applications:
 - - Information extraction from documents.
 - - Enhancing search engines with entity-aware capabilities.
 - - Automatically tagging and categorizing large text corpora.

Automated Moderation

- Objective: Classify and moderate user-generated content.
- Applications:
 - - Filtering offensive or inappropriate content on social media platforms.
 - - Ensuring compliance with community guidelines in forums.
 - - Automating comment moderation on news websites.

News Aggregation

- Objective: Categorize and cluster news articles.
- Applications:
 - - Creating personalized news feeds.
 - - Grouping articles by event or story.
 - - Enhancing the discoverability of related news content.

Medical Text Analysis

- Objective: Classify and analyze medical texts and records.
- Applications:
 - - Categorizing clinical notes by disease or condition.
 - - Extracting relevant information from patient records.
 - - Supporting clinical decision-making processes.

Legal Document Classification

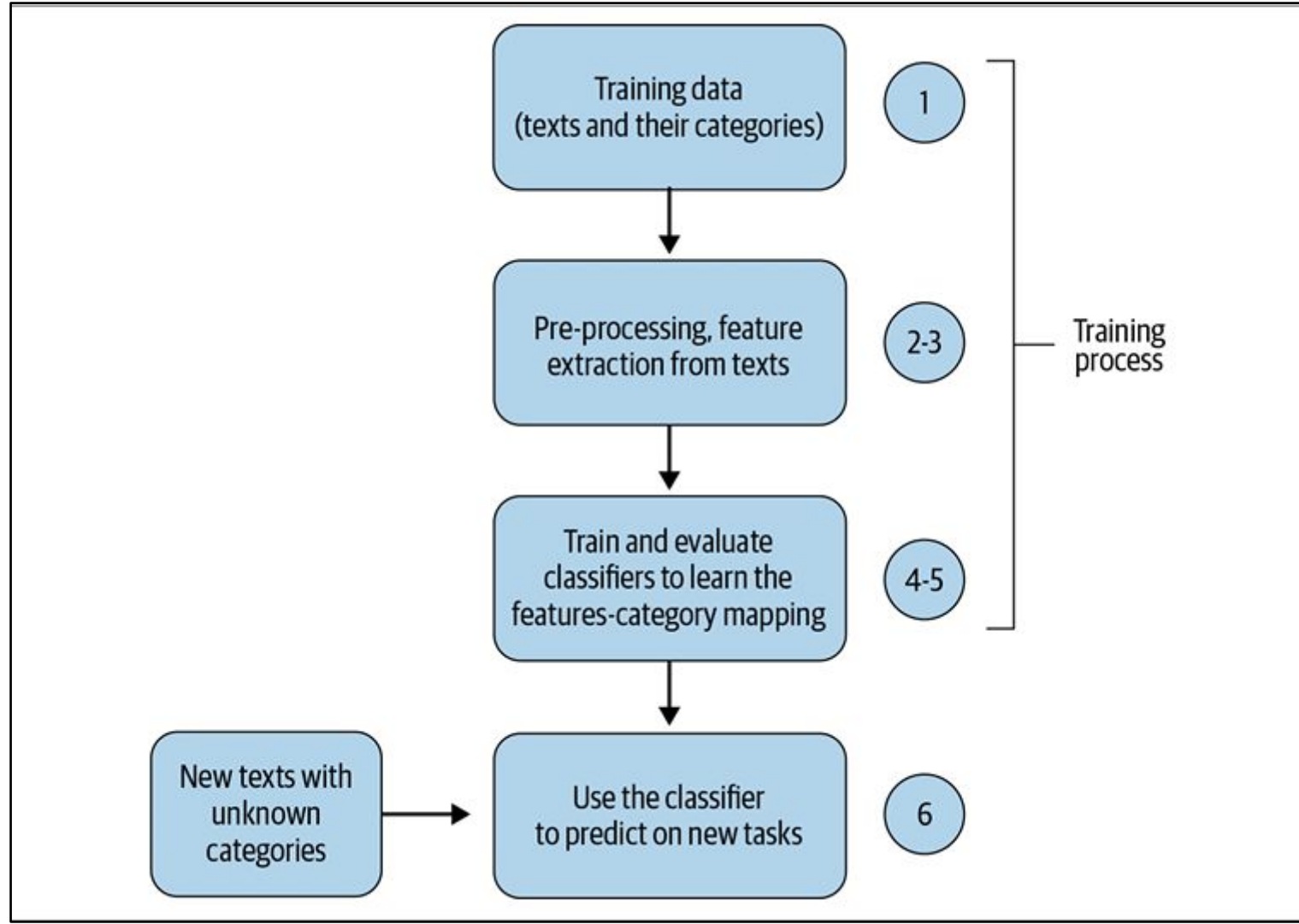
- Objective: Classify legal documents by type, topic, or relevance.
- Applications:
 - - Organizing case files and legal precedents.
 - - Assisting in legal research.
 - - Automating the review of legal documents.

Opinion Mining

- Objective: Extract and classify opinions from text.
- Applications:
 - - Market research to understand consumer opinions.
 - - Political analysis to gauge public opinion.
 - - Brand management to track opinions on products or services.

Steps in Text Classification

- **Data Collection:** Gather a labeled dataset for training the model.
- **Preprocessing:** Clean and preprocess the text data (e.g., tokenization, stopwords removal, stemming/lemmatization).
- **Feature Extraction:** Convert text into numerical features using methods like Bag of Words, TF-IDF, or word embeddings.
- **Model Training:** Train a chosen classifier on the processed data.
- **Evaluation:** Assess the model's performance using metrics such as accuracy, precision, recall, and F1-score.
- **Prediction:** Use the trained model to classify new text data.



Text Classification using Naïve Bayes Classifier

Introduction to Naïve Bayes Classifier

- Naïve Bayes is a family of probabilistic algorithms based on **applying Bayes' theorem with strong independence assumptions between the features**.
- It's widely used in text classification problems due to its simplicity, efficiency, and often surprisingly good performance.

Bayes' Theorem

- Bayes' theorem describes the probability of an event, based on prior knowledge of conditions that might be related to the event.
- **Formula:** $P(A|B) = [P(B|A) * P(A)] / P(B)$
 - $P(A|B)$: Posterior probability of class A given predictor B.
 - $P(B|A)$: Likelihood probability of predictor B given class A.
 - $P(A)$: Prior probability of class A.
 - $P(B)$: Prior probability of predictor B.

- **'Naïve' Assumption:** Assumes that the features in a dataset are independent of each other.
- This is rarely true in real-world scenarios, but the algorithm performs surprisingly well despite this assumption.
- **Independence Assumption:** Each feature contributes independently to the probability of a particular outcome.

Types of Naïve Bayes Classifiers

- **Multinomial Naïve Bayes:** Suitable for discrete data like word counts in text classification.
- **Bernoulli Naïve Bayes:** Suitable for binary/boolean features.
- **Gaussian Naïve Bayes:** Suitable for continuous data and assumes that the features follow a normal distribution.

Multinomial Naïve Bayes for Text Classification

- Multinomial Naïve Bayes is particularly suited for text classification where the **features are the frequencies of the words** present in the document.
- The probability of a document belonging to a class is calculated by multiplying the probabilities of the words (features) in the document given the class.

Detailed Step-by-Step Example:

Sentiment Analysis from reviews

1. Collecting Text Data

we need to gather the text data that we want to classify.

For this example, we'll use a small dataset with text samples labeled as either positive or negative.

```
data = {  
    'text': [  
        'I love this movie',  
        'This is a great product',  
        'I hate this place',  
        'This is the worst service',  
        'I am very happy with the results',  
        'This is terrible'  
    ],  
    'label': ['positive', 'positive', 'negative', 'negative', 'positive', 'negative']  
}
```

```
# Convert to DataFrame
```

```
import pandas as pd
```

```
df = pd.DataFrame(data)
```



2. Preprocessing the Text Data

- Preprocessing involves cleaning and transforming the raw text data into a format suitable for model training.
- Common steps include converting text to lowercase, removing stop words, tokenization, and vectorization.

3. Splitting the Data

- split the dataset into training and test sets.
- The training set is used to train the model, while the test set is used to evaluate its performance.

```
# Split the data into training and test sets  
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(df['text'], df['label'], test_size=0.2, random_state=42)
```

4. Vectorizing the Text Data

- Machine learning models cannot process raw text directly.
- We need to convert the text data into numerical features.
- One common method is using a **CountVectorizer**, which converts text into a matrix of token counts.

```
# Convert text data to numeric data using CountVectorizer
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
X_train_counts = vectorizer.fit_transform(X_train)
X_test_counts = vectorizer.transform(X_test)
```

5. Training the Naïve Bayes Classifier

- We use the **Multinomial Naïve Bayes classifier** for this task.
- We fit the model using the training data.

```
# Train the Naïve Bayes classifier
from sklearn.naive_bayes import MultinomialNB
nb_classifier = MultinomialNB()
nb_classifier.fit(X_train_counts, y_train)
```

6. Making Predictions

- With the trained model, we can make predictions on the test data.

```
# Make predictions on the test data  
y_pred = nb_classifier.predict(X_test_counts)
```


7. Evaluating the Model

- Finally, we evaluate the model's performance using metrics such as accuracy, precision, recall, and F1-score.

```
# Evaluate the model
from sklearn.metrics import accuracy_score, classification_report
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=['negative', 'positive'])

print(f'Accuracy: {accuracy}')
print('Classification Report:')
print(report)
```